



deploy
scale
manage

VERIQTA



container
orchestration
made simple

KUBERNETES

FROM SCRATCH

HANDBOOK

FROM BEGINNER TO PRODUCTION

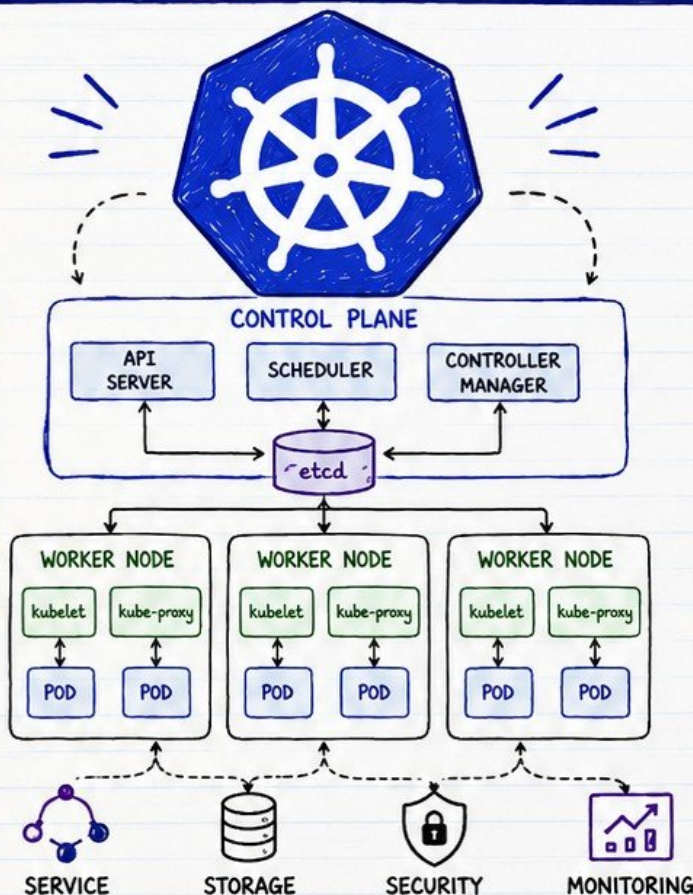
LEARN KUBERNETES
FROM THE GROUND UP

- ✓ Core Concepts
- ✓ Architecture
- ✓ Hands-on Labs
- ✓ Production Best Practices
- ✓ Troubleshooting
- ✓ Security
- ✓ Monitoring
- ✓ CI/CD & GitOps
- ✓ Real-World Scenarios
- ✓ Interview Preparation

★ 40 PAGES

COMPLETE CURRICULUM
PRACTICAL • HANDS-ON
PRODUCTION READY

Build.
Deploy.
Scale.
Repeat.



FROM
SCRATCH
TO
PRODUCTION

WHAT YOU'LL LEARN



KUBERNETES
FUNDAMENTALS



WORKLOADS
& SERVICES



STORAGE
& NETWORKING



SECURITY
& RBAC



MONITORING
& LOGGING



AUTOSCALING
& HA



PRODUCTION
BEST PRACTICES



TROUBLESHOOTING
& INTERVIEWS

MASTER KUBERNETES. BUILD RELIABLE SYSTEMS. RUN PRODUCTION AT SCALE.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



1. INTRODUCTION TO KUBERNETES

“Kubernetes is the operating system for containers in the cloud.”

WHAT YOU WILL LEARN

- ✓ Core concepts
- ✓ Architecture
- ✓ Hands-on practice
- ✓ Production mindset

WHAT IS KUBERNETES?

- Kubernetes (K8s) is an open-source platform designed to automate **deploying**, **scaling**, and **operating** containerized applications.
- It groups containers into logical units, manages their lifecycle, and ensures the application runs reliably at scale.
- Originally developed by **Google** and now maintained by the **Cloud Native Computing Foundation (CNCF)**.



WHY KUBERNETES EXISTS

- ✓ Handle complex container deployments
- ✓ Automatic scaling and self-healing
- ✓ High availability and zero downtime
- ✓ Efficient resource utilization
- ✓ Abstract infrastructure complexity
- ✓ Consistent environments across teams
- ✓ Faster delivery and easier operations

CONTAINERS VS KUBERNETES



CONTAINERS (e.g., Docker)

- Packages application and dependencies
- Runs on a single host
- You manage containers manually
- No built-in scaling or self-healing
- No load balancing or service discovery
- Good for development and small setup

VS

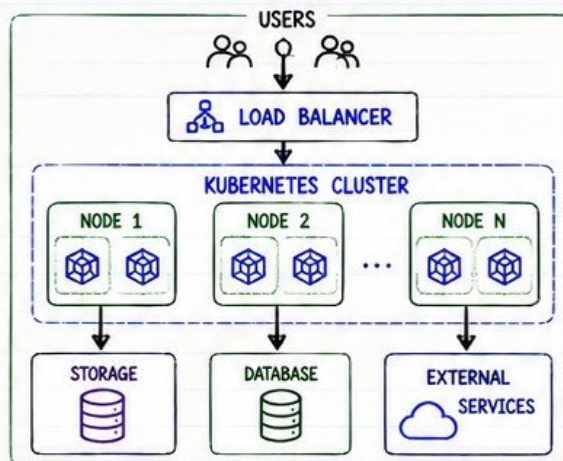


KUBERNETES

- Orchestrates containers across a cluster
- Runs on multiple hosts (nodes)
- Automates deployment, scaling, and updates
- Self-healing (restarts failed containers)
- Built-in load balancing and service discovery
- Built for production and large scale

REAL-WORLD USE CASES

- Cloud-Native Applications
- E-Commerce Platforms
- Microservices Architectures
- AI / ML Workloads
- Data Processing Pipelines
- Enterprise Applications



KEY TAKEAWAYS

- ✓ Kubernetes does not run your application.
- ✓ It runs and manages containers at scale.
- ✓ It provides reliability, scalability and portability.

LEARNING ROADMAP



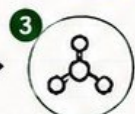
Fundamentals

- Concepts
- Architecture
- Basic Objects



Hands-On Basics

- kubectl
- Pods, Services
- Deployments



Intermediate

- ConfigMaps, Secrets
- Storage
- Ingress, Networking



Advanced

- Scaling, HPA
- Monitoring
- Security, RBAC



Production

- HA, Backups
- Troubleshooting
- Best Practices



Mastery

- System Design
- Real-world Projects
- Continuous Learning



kubernetes

VERIQTA

2. KUBERNETES ARCHITECTURE

PAGE 2

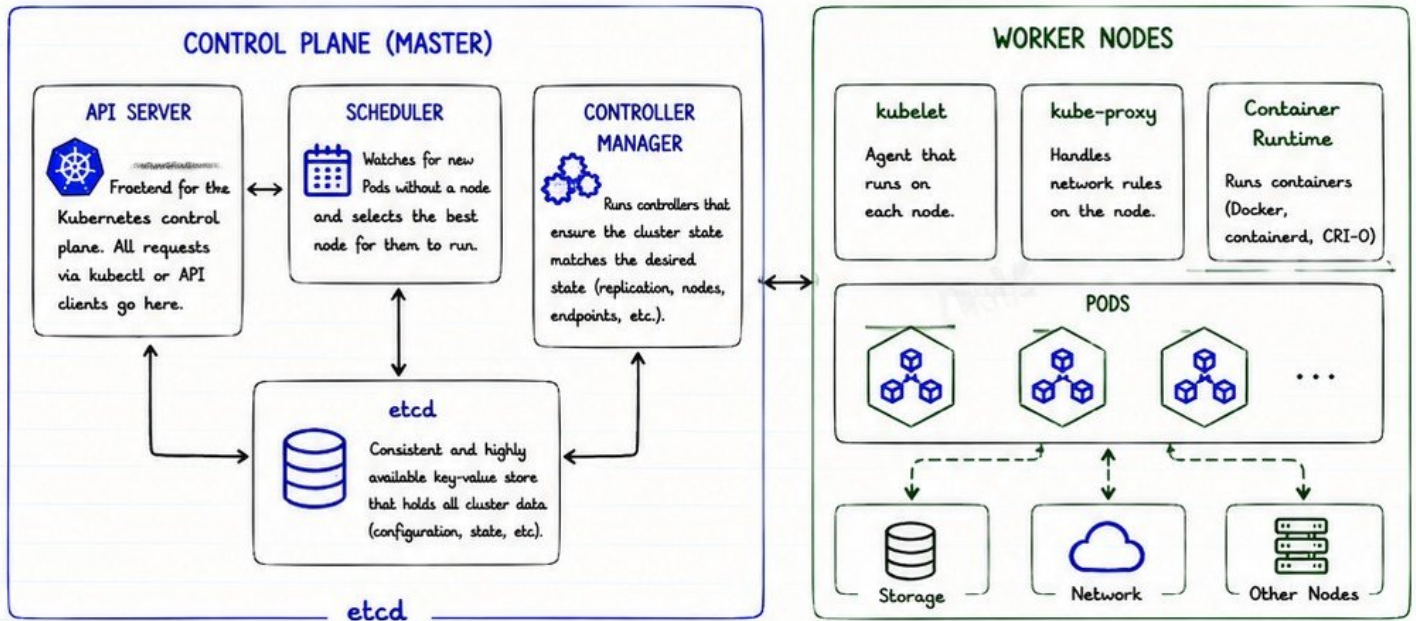
AT A GLANCE

- ✓ Two main parts
- ✓ Control Plane
- ✓ Worker Nodes
- ✓ Work together
- ✓ Desired state achieved

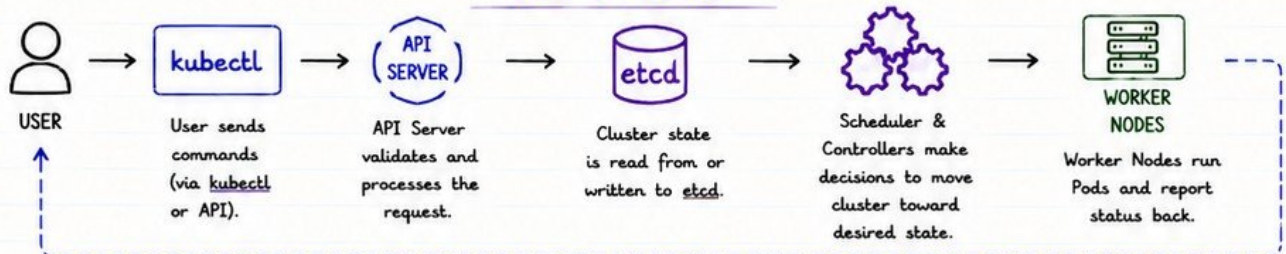
“

Kubernetes follows a **Master (Control Plane) - Worker (Node)** architecture. The **Control Plane** makes decisions about the cluster (scheduling, scaling, healing). **Worker Nodes** run your applications.

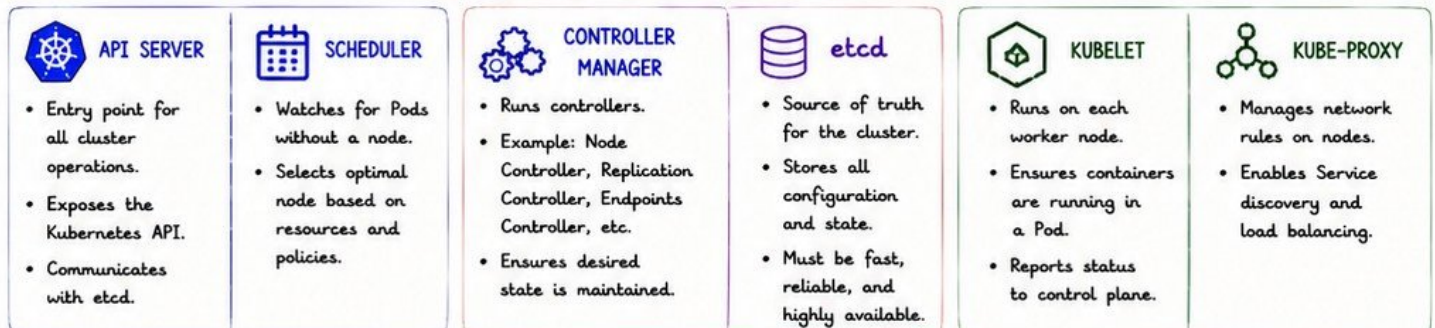
”



HOW IT WORKS



KEY COMPONENTS EXPLAINED



★ **Note:**

The control plane does not run your application containers. It only manages the cluster. The worker nodes run your workloads.



KEY PRINCIPLE

Kubernetes continuously works to ensure the actual state matches the desired state you define.





3. INSTALLING KUBERNETES

“

There are many ways to run Kubernetes.
Choose the right tool for learning and for production.”

”

AT A GLANCE

- ✓ Local: Minikube, Kind
- ✓ Production: kubeadm
- ✓ Control with: kubectl
- ✓ Always verify your cluster

1. MINIKUBE (LOCAL SINGLE NODE)

- Easiest way to run Kubernetes locally.
- Runs a single-node cluster inside a VM.
- Great for beginners and testing.



Install Minikube

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

```
# Start cluster
minikube start
```

```
# Check status
minikube status
```



TIP

Use: minikube dashboard to open the Kubernetes dashboard.

2. KIND (LOCAL MULTI-NODE)

- Runs Kubernetes using Docker containers as nodes.
- Supports multi-node clusters.
- Great for CI/CD and advanced testing.



Install Kind

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.22.0/kind-linux-amd64
chmod +x ./kind && sudo mv ./kind /usr/local/bin/kind
```

Create cluster (multi-node example)

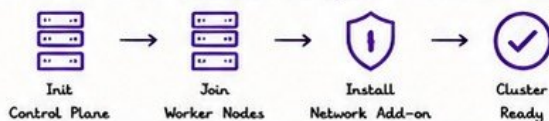
```
kind create cluster --name demo --config - <<EOF
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
- role: worker
EOF
```

```
# Check nodes
kubectl get nodes
```

3. KUBEDRM OVERVIEW (PRODUCTION/GROUP)

- kubeadm helps bootstrap production-ready clusters.
- Creates secure clusters using best practices.
- Requires 1 control plane + worker nodes.

- kubeadm workflow -



Initialize control plane

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

Join worker node

```
kubeadm join <CONTROL_PLANE_IP>:6443 --token <TOKEN> \
--discovery-token-ca-cert-hash sha256:<HASH>
```

4. KUBECTL INSTALLATION

- kubectl is the command-line tool to talk to Kubernetes.
- Works with any Kubernetes cluster.
- Download the latest stable version.



Download kubectl

```
curl -LO "https://dl.k8s.io/release/stable.txt"
curl -LO "https://dl.k8s.io/release/$(cat stable.txt)/bin/linux/amd64/kubectl"
```

```
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

Verify installation

```
kubectl version --client
```

5. VERIFYING THE CLUSTER

- Always verify your cluster after installation.
- Confirm components are running.
- Ensure nodes are Ready.



Check nodes

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
control-plane	Ready	control-plane	2m	v1.30.0
worker-1	Ready	<none>	2m	v1.30.0

Check all pods in all namespaces

```
kubectl get pods -A
```

NAMESPACE	NAME	STATUS	AGE
kube-system	coredns-5d78c9869d-xxxxx	Running	2m
kube-system	kube-apiserver-control-plane	Running	3m
...			

Check cluster info

```
kubectl cluster-info
```

Kubernetes control plane is running at https://127.0.0.1:6443
CoreDNS is running at https://127.0.0.1:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

Check system components

```
kubectl get componentstatuses
```

NAME	STATUS	MESSAGE	ERROR
scheduler	Healthy	ok	
controller-manager	Healthy	ok	
etcd-0	Healthy	{ "health": "true" }	



NOTE:

For learning: use Minikube or Kind.
For production: use kubeadm, managed Kubernetes (EKS, GKE, AKS) or enterprise platforms.



GOAL:

A running, healthy Kubernetes cluster ready for workloads.



NEXT:

Learn core objects: Pods, Deployments, Services and more.



4. UNDERSTANDING KUBECTL

“Kubectl is the command-line interface for interacting with Kubernetes clusters.”

AT A GLANCE

- ✓ Powerful CLI for Kubernetes
- ✓ Manage resources with ease
- ✓ Inspect, debug and automate
- ✓ Essential for every engineer

1. CLUSTER INFORMATION

- Get information about the cluster and nodes.



kubectl cluster-info	# Basic cluster information
kubectl cluster-info dump	# Detailed cluster info
kubectl get nodes	# List all nodes
kubectl get nodes -o wide	# Nodes with more details
kubectl get componentstatuses	# Check core components
kubectl get cs	# Short name for above
kubectl top nodes	# Resource usage (needs metrics-server)

2. NAMESPACES

- Namespaces help isolate and organize resources.



kubectl get namespaces	# List all namespaces
kubectl get ns	# Short name
kubectl create namespace <name>	# Create namespace
kubectl delete namespace <name>	# Delete namespace
kubectl config set-context --current --namespace=<name>	# Set default namespace
kubectl get all -n <namespace>	# Resources in a namespace
kubectl explain namespace	# Show namespace docs

3. RESOURCE MANAGEMENT

- Create, update, delete and manage Kubernetes resources.



kubectl get all	# All resources in current namespace
kubectl get <resource>	# Get specific resource (pods, svc, etc.)
kubectl get <resource> -o wide	# Wide output
kubectl describe <resource> <name>	# Detailed info about a resource
kubectl create -f <file.yaml>	# Create from YAML file
kubectl apply -f <file.yaml>	# Apply changes to resources
kubectl delete -f <file.yaml>	# Delete resources from YAML file
kubectl delete <resource> <name>	# Delete a resource
kubectl edit <resource> <name>	# Edit resource in default editor
kubectl replace -f <file.yaml>	# Replace resource by YAML

4. YAML INSPECTION

- Inspect and validate YAML files and resources.



kubectl get <resource> <name> -o yaml	# Output YAML
kubectl get <resource> <name> -o json	# Output JSON
kubectl explain <resource>	# Explain resource fields
kubectl explain <resource>.<field>	# Explain specific field
kubectl diff -f <file.yaml>	# Show changes before apply
kubectl apply --dry-run=client -f <file.yaml>	# Validate YAML locally
kubectl apply --dry-run=server -f <file.yaml>	# Validate with API server

5. COMMON KUBECTL COMMANDS CHEAT SHEET

	View Resources →	kubectl get pods,svc,deploy,ns kubectl get all	# Get resources # All resources
	Filter & Find →	kubectl get pods -l app=nginx kubectl get pods --field-selector=status.phase=Running	# By label # By field
	Sorting →	kubectl get pods --sort-by=.metadata.name kubectl get pods --sort-by=.status.startTime	# Sort by name # Sort by start time
	Output Format →	kubectl get pods -o wide kubectl get pods -o yaml kubectl get pods -o jsonpath='{.items[*].metadata.name}'	# Wide output # YAML output # JSONPath output
	Logs & Exec →	kubectl logs <pod-name> kubectl logs -f <pod-name> kubectl exec -it <pod-name> -- /bin/bash	# Pod logs # Follow logs # Exec into pod
	Port Forward →	kubectl port-forward <pod-name> 8080:80 kubectl port-forward svc/<service-name> 8080:80	# Forward pod port # Forward service port

TIPS

- ✓ Use tab completion to save time.
kubectl get po<TAB>
- ✓ Check current context:
kubectl config current-context
- ✓ Switch context:
kubectl config use-context <name>
- ✓ Get help for any command:
kubectl --help
kubectl <command> --help

BEST PRACTICES

- ✓ Always work in the right namespace.
- ✓ Use apply -f for declarative management.
- ✓ Use labels to organize and select resources.
- ✓ Inspect resources before making changes.
- ✓ Keep your kubectl and cluster versions in sync.

“ A Pod is the smallest deployable unit in Kubernetes. It represents a single instance of a running process in your cluster. ”

AT A GLANCE

- ✓ One or more containers
- ✓ Share network & storage
- ✓ Deployed on a single node
- ✓ Ephemeral by design

1. POD LIFECYCLE



- **Pending:** Pod is accepted by the system.
- **Running:** Containers are running.
- **Succeeded:** All containers terminated successfully.
- **Failed:** At least one container failed.
- **Unknown:** State cannot be determined.

2. SINGLE-CONTAINER PODS

- Most common type of Pod.
- One container that runs a single process.
- Example: Nginx web server.

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
  
```

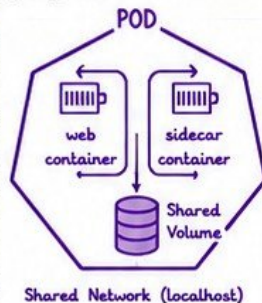


3. MULTI-CONTAINER PODS

- Multiple containers work together in the same Pod.
- They share the same network namespace and storage volumes.
- Use case: Sidecar pattern, log shipping, proxy, etc.

```

apiVersion: v1
kind: Pod
metadata:
  name: web-with-sidecar
spec:
  containers:
    - name: web
      image: nginx:1.25
      ports:
        - containerPort: 80
    - name: sidecar
      image: busybox
      command:
        ["sh", "-c", "-s", "tail -f /var/log/access.log"]
  
```



4. INIT CONTAINERS

- Run before app containers start.
- Useful for setup, migrations, waiting for dependencies.
- Must complete successfully before app containers start.

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-with-init
spec:
  initContainers:
    - name: init-db
      image: busybox
      command: ["sh", "-c", "until nslookup mydb; do echo 'waiting for db'; sleep 2; done"]
  containers:
    - name: app
      image: myapp:1.0
  
```

Init Container(s)
(Run to completion)

App Container(s)
(Start after init)

5. POD BEST PRACTICES



Keep Pods small
Run one main process per container. Use multi-container Pods only when containers are tightly coupled.



Pods are ephemeral
Pods can be created, destroyed or rescheduled at any time.



Use labels
Always add meaningful labels for selection and organization.



Use volumes for persistence
Pod storage is ephemeral. Use Persistent Volumes for data that must survive Pod restarts.



Add health checks
Configure liveness and readiness probes for reliability.



Define resource requests & limits
Set CPU and memory requests and limits to ensure stability.



Run as non-root
Use securityContext to run containers as non-root users.



Centralize logs
Send logs to a central system (ELK, Loki, CloudWatch, & zecce.).



Keep images updated
Use trusted, minimal images and regularly update your containers.



NOTE

A Pod is scheduled and runs on a single node. If the node fails, the Pod is recreated on another healthy node (by controllers like Deployment).



KEY TAKEAWAY

Pods are the building blocks of Kubernetes. Design them to be simple, stateless (when possible), resilient and observable.



COMMON COMMANDS

```

kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- sh
  
```

6. LABELS, SELECTORS AND ANNOTATIONS

“ Labels are for identifying objects.
Selectors are for finding objects.
Annotations are for adding information. ”

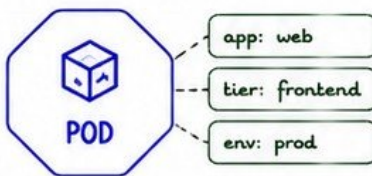
AT A GLANCE

- ✓ Labels = Identify
- ✓ Selectors = Find
- ✓ Annotations = Inform
- ✓ Powerful for organization
- ✓ Essential for production

1. LABELS

- Key/value pairs attached to Kubernetes objects.
- Used to identify and organize resources.
- Can be used on Pods, Services, Deployments, Nodes, etc.

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
    tier: frontend
    env: prod
```



2. MATCH SELECTORS

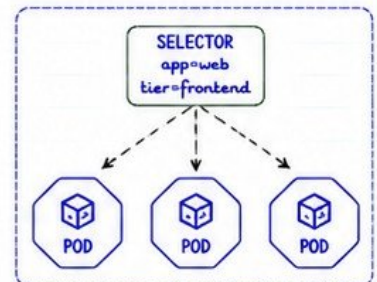
- Selectors are used to find and group resources
- Based on label key/value pairs.
- Used by Deployments, Services, NetworkPolicies, etc.

Equality-based selector

```
selector:
  matchLabels:
    app: web
    tier: frontend
```

Set-based selector

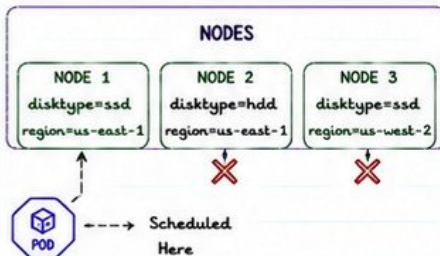
```
selector:
  matchExpressions:
    - key: env
      operator: In
      values: ["prod", "staging"]
```



3. NODE SELECTORS

- Used to schedule Pods on specific nodes.
- Based on node labels.
- Simple but effective for basic scheduling.

```
apiVersion: v1
kind: Pod
metadata:
  name: db-pod
spec:
  nodeSelector:
    disktype: ssd
    region: us-east-1
```



4. ANNOTATIONS

- Key/value pairs used to store non-identifying information.
- Not used for selection.
- Useful for notes, tooling, integrations, and metadata.

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  annotations:
    description: "Web frontend pod"
    owner: "platform-team"
    backup.velero.io/backup: "true"
    prometheus.io/scrape: "true"
    prometheus.io/port: "8080"
```

ANNOTATIONS DO NOT AFFECT HOW OBJECTS ARE SELECTED.

5. PRODUCTION USAGE

ORGANIZATION

- Group resources by app, team, environment.
- Example:
 - app: payment
 - team: platform
 - env: prod

SERVICE DISCOVERY

- Services use selectors to route traffic to Pods.
- Example:


```
selector:
  app: api
  tier: backend
```

ENVIRONMENT MANAGEMENT

- Isolate resources by environment.
- Example:
 - env: dev
 - env: staging
 - env: prod

MONITORING & TOOLS

- Tools use labels/annotations to discover and monitor.
- Example annotation:


```
prometheus.io/scrape: "true"
prometheus.io/port: "8080"
```

AUTOMATION & POLICIES

- Use labels in NetworkPolicies, RBAC, Backup, etc.
- Example:


```
matchLabels:
  app: web
```

BEST PRACTICES

- ✓ Use consistent label keys across your environment.
- ✓ Follow a clear label naming convention.
- ✓ Do not use labels for unique IDs.
- ✓ Keep labels simple, small and meaningful.
- ✓ Document your label taxonomy.

COMMON LABEL KEYS (EXAMPLES)

KEY	PURPOSE	KEY	PURPOSE
app	Application name	team	Owning team
tier	Layer (frontend/backend/db)	version	Version of the application
env	Environment (dev/staging/prod)	component	Component name



kubernetes

VERIQTA

7. NAMESPACES

PAGE 7

“

Namespaces provide a way to divide cluster resources between multiple users, teams, or projects.

They are like virtual clusters inside a physical cluster.”

AT A GLANCE

- ✓ Default namespaces exist
- ✓ Create namespaces easily
- ✓ Isolate resources
- ✓ Organize by project/team
- ✓ Essential for scale

1. DEFAULT NAMESPACES

- Kubernetes comes with several default namespaces.



NAMESPACE	PURPOSE
default	Default namespace for users and applications.
kube-system	Kubernetes system components.
kube-public	Cluster-wide resources that should be public.
kube-node-lease	Leases for node heartbeats.

List all namespaces
kubectl get namespaces

NAME	STATUS	AGE
default	Active	10d
kube-system	Active	10d
kube-public	Active	10d
kube-node-lease	Active	10d

2. CREATING NAMESPACES

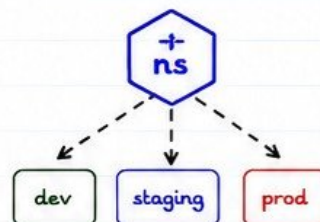
- Create namespaces using YAML or kubectl.

Using kubectl
kubectl create namespace dev

Using YAML
apiVersion: v1
kind: Namespace
metadata:
 name: dev
 labels:
 environment: development
 owner: platform-team

Verify
kubectl get namespace dev

NAME	STATUS	AGE
dev	Active	2m



Namespaces are scoped resources. Objects in one namespace cannot see objects in another namespace.

3. ISOLATION

- Resources in one namespace are isolated from others.
- The same resource names can exist in different namespaces.

dev namespace



prod namespace



Isolated from each other

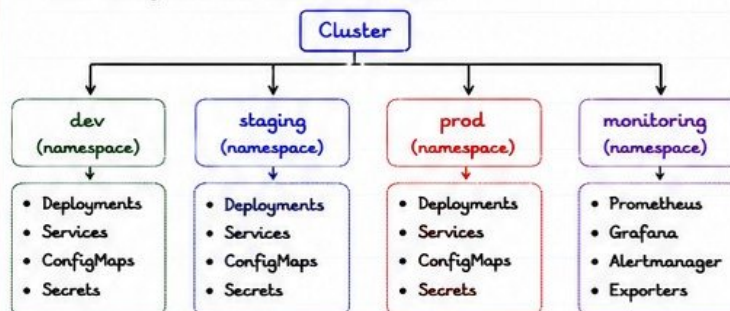
Example: same pod name, different namespaces

kubectl get pods api -n dev

kubectl get pods api -n prod

4. RESOURCE ORGANIZATION

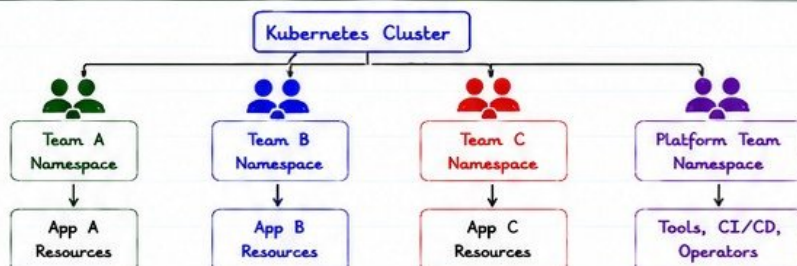
- Group resources by project, application, environment, or department.
- Makes management and access control easier.



5. MULTI-TEAM ENVIRONMENTS

- Each team gets their own namespace.
- Teams manage their resources without affecting others.
- RBAC can be applied per namespace.

Example structure
team-a namespace -> app-a
team-b namespace -> app-b
platform namespace -> CI/CD, tools
shared namespace -> shared services



BEST PRACTICES

- ✓ Use namespaces for all environments (dev, staging, prod).
- ✓ Never deploy workloads in the default namespace.
- ✓ Apply ResourceQuotas and LimitRanges per namespace.
- ✓ Use meaningful namespace names and labels.
- ✓ Control access using RBAC per namespace.

COMMON COMMANDS

```

kubectl get namespaces
kubectl create namespace <name>
kubectl delete namespace <name>
kubectl get all -n <namespace>
kubectl describe namespace <name>
  
```

KEY TAKEAWAY

Namespaces are the foundation of multi-tenancy in Kubernetes. Use them to isolate, organize and scale your applications like a production engineer.

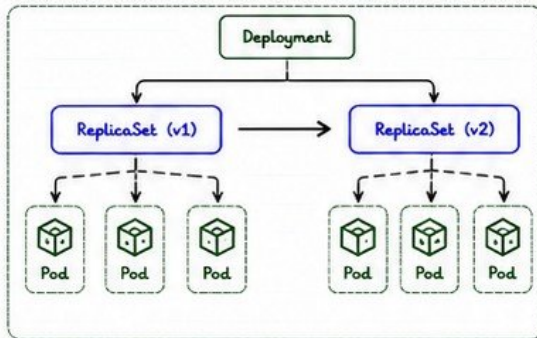
“Deployments provide declarative updates for Pods and ReplicaSets. They allow you to roll out changes, roll back if needed, and scale your applications with ease.”

AT A GLANCE

- ✓ Manage application updates
- ✓ Ensure desired state
- ✓ Zero-downtime deployments
- ✓ Easy rollbacks
- ✓ Scale applications
- ✓ Multiple deployment strategies

1. DEPLOYMENTS OVERVIEW

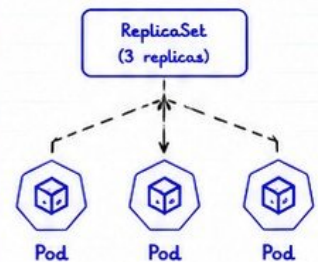
- Deployments manage ReplicaSets and Pods.
- They ensure the desired number of Pod replicas are running.
- When you update a Deployment, a new ReplicaSet is created.
- Old ReplicaSet is scaled down automatically.



2. REPLICASETS

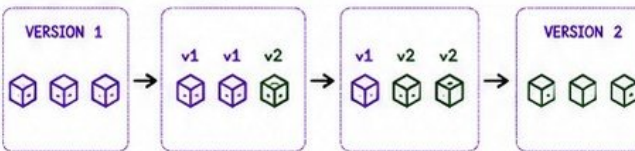
- A ReplicaSet ensures a specified number of Pod replicas are running.
- Usually managed by a Deployment (not directly by you).
- If a Pod fails, the ReplicaSet creates a new one.
- Each Deployment revision has its own ReplicaSet.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.25
```



3. ROLLING UPDATES

- Default strategy used by Deployments.
- New Pods are created with the new version.
- Old Pods are terminated gradually.
- Zero downtime during updates.



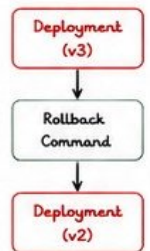
4. ROLLBACKS

- Easily revert to a previous ReplicaSet.
- Instant rollback with kubectl.
- Keeps your application stable and safe.

```
# View rollout history
kubectl rollout history deployment <name>

# Rollback to previous revision
kubectl rollout undo deployment <name>

# Rollback to specific revision
kubectl rollout undo deployment <name> --to-revision=2
```



5. SCALING

- Scale up or down the number of Pod replicas easily.
- Deployment creates or removes Pods automatically.

```
# Scale to 5 replicas
kubectl scale deployment web --replicas=5

# Check current replicas
kubectl get deployment web
```



6. DEPLOYMENT STRATEGIES

- Control how updates are rolled out.
- Different strategies for different needs.

STRATEGY	DESCRIPTION	USE CASE
RollingUpdate	Default. Gradually replaces old Pods with new Pods.	Most applications. Zero downtime.
Recrreate	Terminates all old Pods before creating new Pods.	Simple apps. Downtime acceptable.
Blue/Green	Run two environments (blue and green). Switch traffic when ready.	High availability. Safe releases.
Canary	Route a small % of traffic to new version first.	Test with real traffic before full rollout.

7. DEPLOYMENT BEST PRACTICES



Always Use Rolling Updates

Provides zero downtime and smooth user experience.



Monitor Rollouts

Watch your deployments closely during updates.



Test Rollbacks

Always test rollback processes in staging.



Set Resource Limits

Define CPU/Memory requests and limits for stability.



Use Labels Consistently

Makes management and grouping easier.



Keep Images Versioned

Use specific image tags, avoid :latest.



TIP

Deployments manage the life cycle of your application. Master them to deliver reliable, scalable and production-ready systems.



COMMON COMMANDS

```
kubectl get deployments
kubectl describe deployment <name>
kubectl rollout status deployment <name>
kubectl rollout history deployment <name>
```



KEY TAKEAWAY

Deployments help you release features faster, safer and with confidence. They are the backbone of production application delivery in Kubernetes.

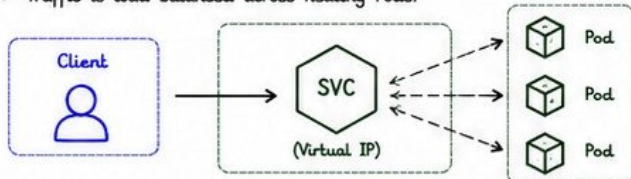
“ Services provide a stable network endpoint for accessing Pods. They abstract Pod IPs and enable discovery, load balancing, and secure access. ”

AT A GLANCE

- ✓ Expose applications reliably
- ✓ Multiple service types
- ✓ Stable network identity
- ✓ Internal & external access
- ✓ Essential for microservices

1. WHAT IS A SERVICE?

- A Service is an abstraction that defines a logical set of Pods and a policy to access them.
- It provides a stable IP address and DNS name.
- Traffic is load balanced across healthy Pods.



Service DNS: <service-name>.<namespace>.svc.cluster.local
Example: web-service.default.svc.cluster.local

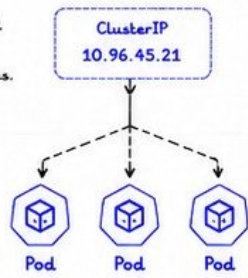
2. SERVICE TYPES OVERVIEW

TYPE	SCOPE	EXPOSED	USE CASE
ClusterIP	Internal	Cluster internal IP	Default type. Access within the cluster.
NodePort	External (Port on Nodes)	<NodeIP>:<Port>	Access via node IP on a static port.
LoadBalancer	External (Cloud)	Cloud LB IP	Cloud provider LB provisions a public IP.
ExternalName	External DNS	CNAME to external name	Map service to external DNS.

3. CLUSTERIP (DEFAULT)

- Exposes the Service on a cluster-internal IP.
- Only accessible from within the cluster.
- Ideal for backend services and microservices.

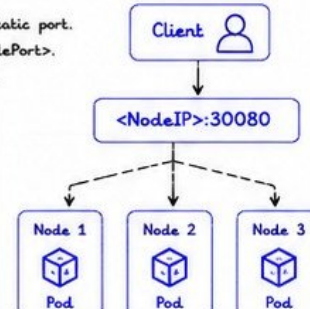
```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - port: 80      # Service port
      targetPort: 8080 # Pod port
```



4. NODEPORT

- Exposes the Service on each Node's IP at a static port.
- Accessible from outside using <NodeIP>:<NodePort>.
- Useful for on-prem environments and testing.

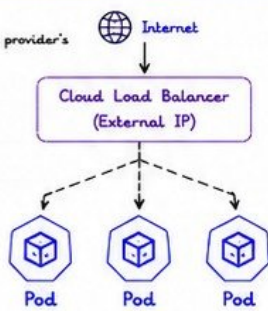
```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080 # Range: 30000-32767
```



5. LOADBALANCER

- Exposes the Service externally using a cloud provider's Load Balancer.
- Automatically provisions a public IP/DNS.
- Best for production applications.

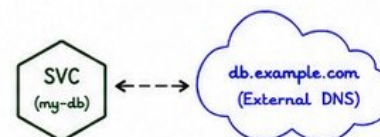
```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
```



6. EXTERNALNAME

- Maps the Service to an external DNS name (CNAME record).
- No proxying or load balancing, just DNS resolution.

```
apiVersion: v1
kind: Service
metadata:
  name: my-db
spec:
  type: ExternalName
  externalName: db.example.com
  ports:
    - port: 5432
```



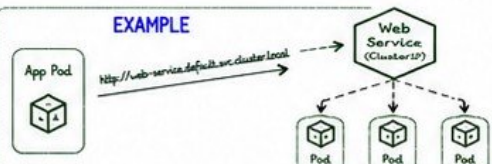
7. SERVICE DISCOVERY

- Kubernetes automatically creates DNS records for Services.
- Applications can reach Services by name, not IP.
- Works across namespaces with the right permissions.

SERVICE DNS FORMAT

Format	Description
<service>	Same namespace
<service>.<namespace>	Specific namespace
<service>.<namespace>.svc	Cluster-wide
<service>.<namespace>.svc.cluster.local	Fully qualified domain

EXAMPLE



BEST PRACTICES

- Use ClusterIP for internal communication.
- Use LoadBalancer for production external access.
- Avoid NodePort in cloud environments.
- Use meaningful service names and labels.
- Keep ports consistent across environments.

COMMON COMMANDS

```
kubectl get svc
kubectl describe svc <name>
kubectl get svc -o wide
kubectl expose deployment <name> --type=NodePort --port=80
kubectl expose deployment <name> --type=LoadBalancer --port=80
```

KEY TAKEAWAY

Services provide stable access to Pods, enable load balancing, and are the foundation of communication in Kubernetes applications.

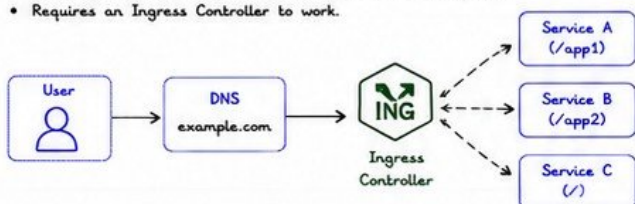
“ Ingress provides HTTP/HTTPS access to services in a cluster. It manages external access, routing, TLS termination, and advanced traffic rules. ”

AT A GLANCE

- ✓ Manage external access to services
- ✓ Host and path based routing
- ✓ TLS/SSL termination
- ✓ Rewrite, redirects and rules
- ✓ Works with Ingress Controllers
- ✓ Essential for production apps

1. INGRESS OVERVIEW

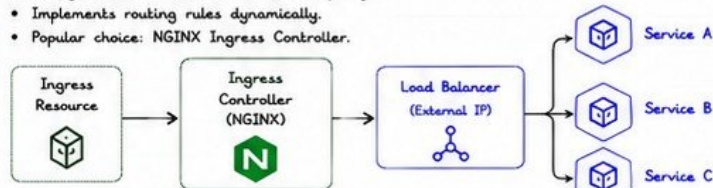
- Ingress exposes multiple Services through a single entry point.
- It uses rules to route traffic based on hostnames and paths.
- Requires an Ingress Controller to work.



Common Ingress Controllers: NGINX Ingress Controller | Traefik | AWS ALB Ingress | HAProxy | GKE Ingress | Istio Gateway (Advanced)

2. INGRESS CONTROLLER

- Watches Ingress resources.
- Configures a load balancer or reverse proxy.
- Implements routing rules dynamically.
- Popular choice: NGINX Ingress Controller.



Ingress Controller Install (NGINX)

```

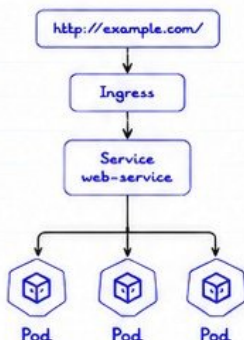
# Add Helm repo
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
# Install
helm install ingress-nginx ingress-nginx/ingress-nginx -n ingress-nginx --create-namespace
  
```

3. HTTP ROUTING (BASIC)

- Route traffic from the root (/) or any path to a Service.
- Uses path rules.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: http-routing
spec:
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
  
```

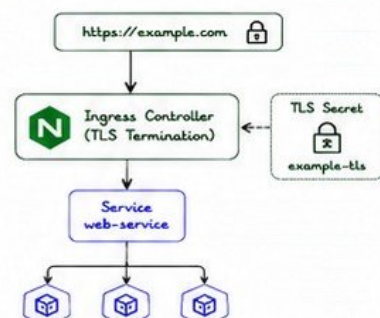


4. TLS (HTTPS) TERMINATION

- Terminate TLS at the Ingress Controller.
- Use TLS secrets for certificates.
- Offloads encryption from backend services.

```

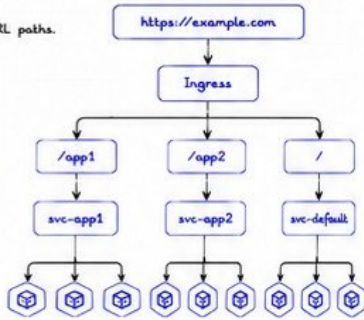
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
spec:
  tls:
  - hosts:
    - example.com
    secretName: example-tls
  rules:
  - host: example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
  
```



5. PATH ROUTING

- Route to different services based on URL paths.
- pathType: Prefix (default) or Exact.

PATH	PATH TYPE	ROUTED TO
/app1	Prefix	service-app1
/app2	Prefix	service-app2
/	Prefix	service-default

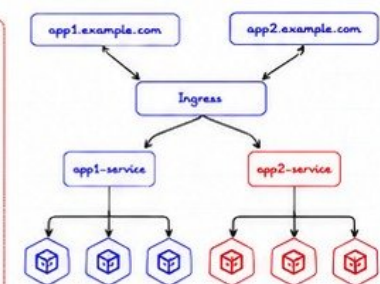


6. HOST-BASED ROUTING

- Route traffic based on hostnames.
- Useful for multi-service, multi-domain apps.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: host-routing
spec:
  rules:
  - host: app1.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app1-service
            port:
              number: 80
  - host: app2.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app2-service
            port:
              number: 80
  
```



7. COMMON ANNOTATIONS (NGINX)

ANNOTATION	PURPOSE
nginx.ingress.kubernetes.io/rewrite-target	Rewrite URL paths
nginx.ingress.kubernetes.io/ssl-redirect	Force HTTPS
nginx.ingress.kubernetes.io/force-ssl-redirect	Redirect HTTP to HTTPS
nginx.ingress.kubernetes.io/proxy-body-size	Set max body size
nginx.ingress.kubernetes.io/timeout-read	Set read timeout

BEST PRACTICES

- ✓ Use Ingress for all external HTTP/HTTPS access.
- ✓ Use TLS everywhere.
- ✓ Keep Ingress rules simple and maintainable.
- ✓ Use meaningful hostnames and paths.
- ✓ Monitor Ingress Controller logs and metrics.
- ✓ Use separate Ingress per environment/team.

COMMON COMMANDS

```

kubectl get ingress
kubectl describe ingress <name>
kubectl get ingress -A
kubectl get ingress -o wide
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller
  
```

KEY TAKEAWAY

Ingress is the gateway to your Kubernetes applications. It brings security, routing, and scalability to production workloads.

11. CONFIGMAPS

“

ConfigMaps store non-sensitive configuration data as key-value pairs. They help you decouple configuration from container images for flexibility and portability.

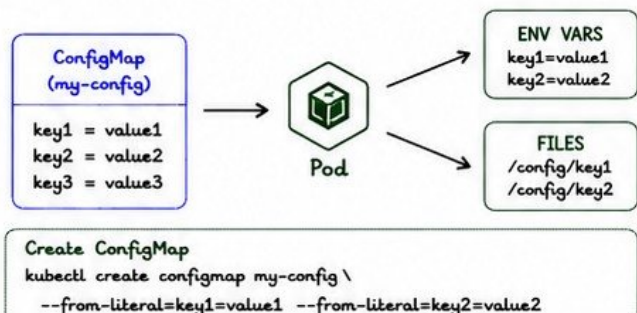
”

AT A GLANCE

- ✓ Store application configuration
- ✓ Use as environment variables
- ✓ Mount as files in containers
- ✓ Update configurations safely
- ✓ Essential for 12-Factor apps

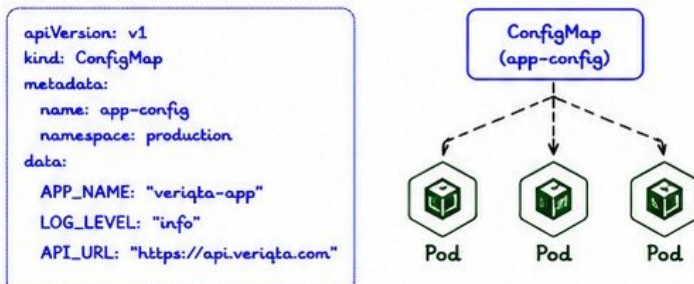
1. CONFIGMAP OVERVIEW

- ConfigMaps store non-sensitive configuration as key-value pairs.
- They can be consumed as environment variables or files.
- Ideal for application config, feature flags, endpoints, etc.



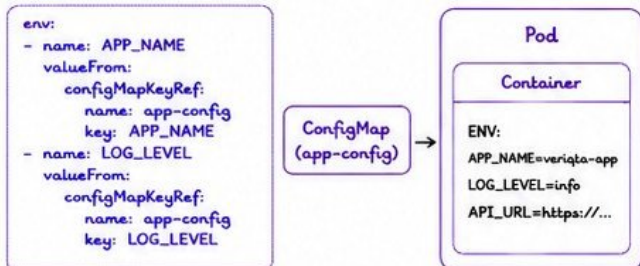
2. CONFIGURATION MANAGEMENT

- Centralize configuration outside of the container image.
- Easy to update without rebuilding images.
- Promote consistency across environments.
- Works across namespaces (with proper RBAC).



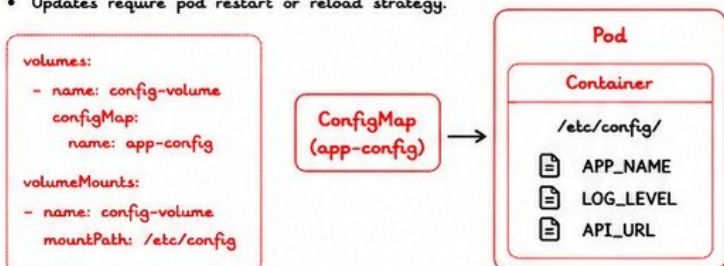
3. ENVIRONMENT VARIABLES

- Inject ConfigMap values as environment variables.
- Perfect for app settings read at startup.



4. MOUNTED FILES

- Mount ConfigMap as files inside the container.
- Great for apps that read config files.
- Updates require pod restart or reload strategy.



5. RELOAD STRATEGIES

- ConfigMap updates do NOT automatically reload in pods.
- Use one of these strategies to apply changes.

STRATEGY	HOW IT WORKS	BEST FOR
Pod Restart	Restart pods after ConfigMap update.	Simple apps
Rolling Restart	kubectl rollout restart deployment/app	Stateless apps
Sidecar Watcher	Sidecar detects changes and signals main container.	Apps supporting reload signals (SIGHUP, etc.)
Reloader Controller	Reloader detects changes and restarts pods automatically.	Kubernetes-native solutions

6. BEST PRACTICES

- ✓ Keep ConfigMaps small and focused.
- ✓ Do not store sensitive data (use Secrets for that).
- ✓ Use meaningful names and clear keys.
- ✓ Version your ConfigMaps using suffixes (e.g., app-config-v1).
- ✓ Document all keys and expected values.
- ✓ Use labels for organization and selection.
- ✓ Combine with Deployments for safe updates.
- ✓ Monitor and validate config at application startup.

COMMON COMMANDS

```

kubectl create configmap my-config --from-literal=key=value Create from literal
kubectl create configmap my-config --from-file=./config Create from file
kubectl get configmaps List ConfigMaps
kubectl describe configmap my-config Show details
kubectl delete configmap my-config Delete ConfigMap
  
```

KEY TAKEAWAY

ConfigMaps help you separate configuration from code, making applications more flexible, portable, and environment-agnostic. Use them to manage settings cleanly and update safely in production.

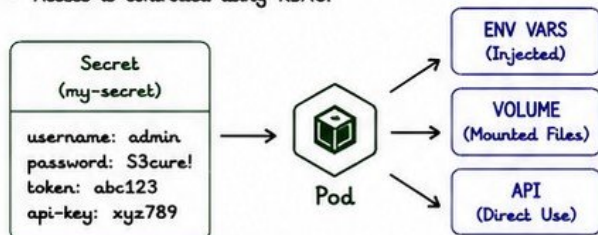
“ Secrets store sensitive information such as passwords, tokens, and keys. They are encoded but not encrypted. Use them to keep sensitive data out of images and code. ”

AT A GLANCE

- ✓ Store sensitive data securely
- ✓ Multiple secret types available
- ✓ Use as environment variables
- ✓ Mount as files in containers
- ✓ Apply strong security practices
- ✓ Essential for production systems

1. SECRET OVERVIEW

- Secrets store sensitive data in key-value pairs.
- Kubernetes encodes Secret data using Base64.
- Not encrypted by default - secure your cluster!
- Access is controlled using RBAC.



2. SECRET TYPES

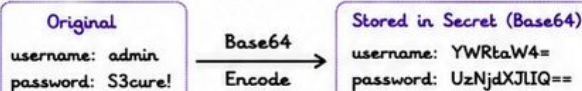
TYPE	DESCRIPTION	USE CASES
Opaque	Arbitrary user-defined key-value data	Passwords, tokens, API keys, config
kubernetes.io/dockerconfigjson	Stores Docker registry credentials	Pull private images from registries
kubernetes.io/tls	Stores TLS certificate and private key	TLS for Ingress, internal services
kubernetes.io/basic-auth	Stores basic auth credentials	Authentication for applications
kubernetes.io/ssh-auth	Stores SSH private key	SSH access to services / systems

3. BASE64 ENCODING

- Kubernetes encodes secret values using Base64.
- Base64 is NOT encryption. Anyone can decode it.
- Use for transport safety, not for security.

Create from literal:

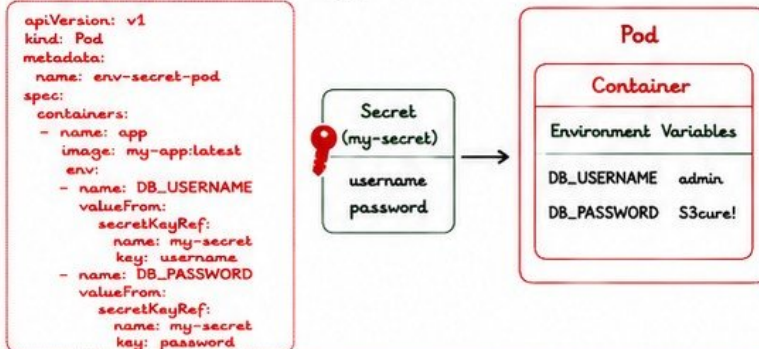
```
kubectl create secret generic my-secret \
  --from-literal=username=admin \
  --from-literal=password=S3cure!
```



Decode: `echo "UzNjdXJlIQ==" | base64 -d → S3cure!`

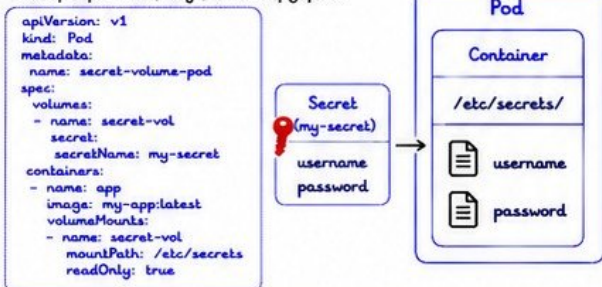
4. ENVIRONMENT VARIABLES

- Inject secret values as environment variables in containers.
- Convenient for apps that read config from env.



5. MOUNTED SECRETS

- Mount secrets as files inside the container.
- Useful for certs, keys, and config files.



6. SECURITY BEST PRACTICES

- ✓ Enable RBAC and restrict secret access.
- ✓ Never commit secrets to Git or images.
- ✓ Use secrets manager (e.g., AWS Secrets Manager, Vault).
- ✓ Encrypt etcd data at rest.
- ✓ Limit who can create, read, update, and delete secrets.
- ✓ Use short-lived tokens and rotate them.
- ✓ Audit secret access regularly.
- ✓ Use dedicated namespaces for sensitive workloads.
- ✓ Avoid exposing secrets in logs or error messages.
- ✓ Use TLS for all communication.



RBAC Example (Read Secret)

```
kubectl create role secret-reader \
  --verb=get,list \
  --resource=secrets
```

```
kubectl create rolebinding secret-reader-binding \
  --role=secret-reader \
  --user=dev-user \
  --namespace=production
```

COMMON COMMANDS

```

kubectl create secret generic my-secret --from-literal=key=value  Create secret from literal
kubectl create secret generic my-secret --from-file=./file       Create secret from file
kubectl get secrets                                              List all secrets
kubectl describe secret my-secret                               Show secret details
kubectl delete secret my-secret                                  Delete a secret
  
```



KEY TAKEAWAY

Secrets are not encrypted by default. They are encoded (Base64) for safe transport. Use strong access controls, external secret managers, and best practices to protect sensitive data in your Kubernetes environment.



13. PERSISTENT STORAGE

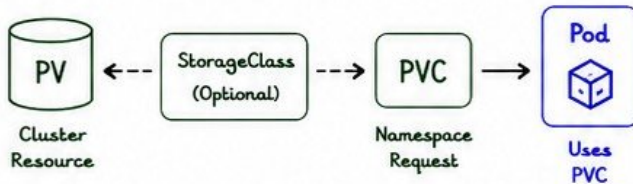
AT A GLANCE

- ✓ Persistent data for stateful apps
- ✓ PVs are cluster resources
- ✓ PVCs are namespace requests
- ✓ StorageClasses enable automation
- ✓ Dynamic provisioning simplifies ops
- ✓ Data survives Pod and node failures

“ Persistent Storage provides durable data for your applications. It survives Pod restarts, rescheduling, and even node failures. Essential for databases, stateful apps, and critical workloads. ”

1. PERSISTENT STORAGE OVERVIEW

- Kubernetes separates storage from Pods.
- PersistentVolumes (PVs) are cluster-wide storage.
- PersistentVolumeClaims (PVCs) are namespace requests.
- PVs outlive Pods. Data remains safe.



Pod does not use PV directly, It uses PVC.
PVC is bound to a suitable PV.

2. PERSISTENT VOLUMES (PVs)

- PVs are pieces of storage in the cluster.
- Created by admin or dynamically by StorageClass.
- Independent of any Pod lifecycle.

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-data
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: fast-ssd
  hostPath:
    path: /data/pv-data
  
```

PV Lifecycle



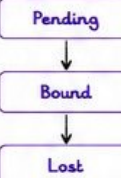
3. PERSISTENT VOLUME CLAIMS (PVCs)

- PVCs are requests for storage by users (Pods).
- They specify size, access mode, and StorageClass.
- PVCs live in a namespace.

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: fast-ssd
  
```

PVC Status



When Bound, the PVC is linked to a PV and can be used by Pods.

4. STORAGECLASSES

- StorageClasses define how storage is provisioned.
- They enable dynamic provisioning of PVs.
- Can set parameters like type, IOPS, throughput, etc.

```

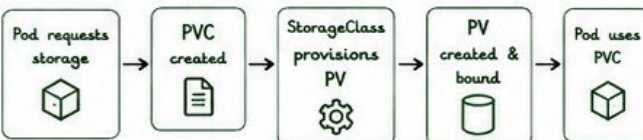
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-efs
parameters:
  type: gp3
  fsType: ext4
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
  
```

Common Provisioners

- kubernetes.io/aws-efs
- kubernetes.io/gce-pd
- kubernetes.io/azure-disk
- rancher.io/local-path
- csi.* (many CSI drivers)

5. DYNAMIC PROVISIONING

- PVC requests storage.
- StorageClass provisions a PV automatically.
- No manual PV creation needed.



Requires a StorageClass with a provisioner that supports dynamic provisioning.

6. STATEFUL STORAGE USE CASES

- Databases (MySQL, PostgreSQL, MongoDB)
- Message brokers (Kafka, RabbitMQ)
- Caching systems (Redis with persistence)
- File servers, object storage, and more



Workload Type	Why Persistent Storage?
Databases	Data must survive restarts and failures
Message Brokers	Queues and logs must persist
Caches	Warm data and snapshots persist
File/Object Storage	Files and objects must be durable

7. BEST PRACTICES

- ✓ Use StorageClasses for dynamic provisioning.
- ✓ Choose correct accessModes for your workload.
- ✓ Set appropriate reclaimPolicy (Retain/Delete).
- ✓ Monitor storage usage and capacity.
- ✓ Use ReadWriteOncePod when supported for extra safety.
- ✓ Back up stateful data regularly.
- ✓ Test failover and node drains.
- ✓ Use dedicated storage for critical workloads.
- ✓ Document storage dependencies.
- ✓ Keep storage IOPS and throughput in mind.

COMMON COMMANDS

```

kubectl get pv
kubectl get pvc
kubectl describe pv <name>
kubectl describe pvc <name>
kubectl get sc
kubectl describe sc <name>
kubectl get pods -o wide
  
```

★ KEY TAKEAWAY

Persistent Storage ensures that your application data is durable, available, and separate from the lifecycle of Pods and nodes. Use StorageClasses and PVCs to manage storage the Kubernetes way.



kubernetes

VERIQTA

PAGE 14

14. RESOURCE MANAGEMENT

“

Resource management ensures your applications get the resources they need without affecting the stability of the cluster. Plan, allocate, and optimize.

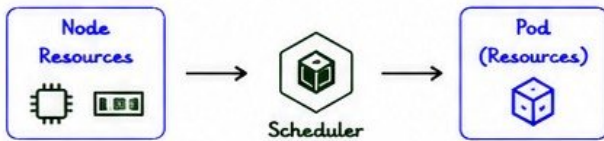
”

AT A GLANCE

- ✓ Set requests for guaranteed resources
- ✓ Set limits to cap maximum usage
- ✓ QoS classes affect Pod eviction priority
- ✓ Balance resources for efficiency
- ✓ Monitor and optimize continuously

1. RESOURCE MANAGEMENT OVERVIEW

- Kubernetes manages CPU and Memory resources.
- You define requests (minimum) and limits (maximum).
- Helps in scheduling, stability, and fair resource distribution.
- Key to performance, cost optimization, and reliability.



If a Pod does not specify requests, scheduler cannot make decisions accurately. Always set requests and limits.

2. CPU & MEMORY REQUESTS

- Requests are the minimum resources guaranteed to the container.
- Used by scheduler to place Pods on nodes.
- Prevent resource contention.

```

apiVersion: v1
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
    - name: app
      image: nginx
      resources:
        requests:
          cpu: "250m"      # 0.25 CPU Core
          memory: "256Mi"  # 256 MiB
  
```

CPU UNITS

1 CPU = 1000m
500m = 0.5 CPU
250m = 0.25 CPU
100m = 0.1 CPU

MEMORY UNITS

1Gi = 1024Mi
512Mi = 0.5Gi
256Mi = 0.25Gi

3. CPU & MEMORY LIMITS

- Limits are the maximum resources a container can use.
- Container is throttled (CPU) or killed (Memory) if limit is exceeded.
- Protects the node and other workloads from resource hogging.

```

resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"      # Max 0.5 CPU
    memory: "512Mi"  # Max 512 MiB
  
```

CPU LIMIT
When exceeded,
CPU is throttled.
Application slows down.

MEMORY LIMIT
When exceeded,
container is OOMKilled
(Out Of Memory).

4. QUALITY OF SERVICE (QOS)

- Kubernetes classifies Pods into QoS classes.
- Affects eviction priority and resource guarantees.

QoS CLASS	CONDITION	GUARANTEE	EVICTON PRIORITY
Guaranteed	requests = limits for CPU & Memory	Highest Strong guarantee	Lowest (evicted last)
Burstable	requests < limits or only requests set	Medium Best-effort within limits	Medium
BestEffort	No requests No limits	None No guarantee	Highest (evicted first)

Check QoS of Pod:

kubectl get pod <pod-name> -o wide

5. RESOURCE OPTIMIZATION

- Right-size your applications.
- Avoid over-provisioning and under-provisioning.
- Monitor, analyze, and adjust regularly.

OVER-PROVISIONING

- Wastes resources
- Higher cost
- Low cluster efficiency



OPTIMAL

- Better performance
- Cost effective
- Stable workloads



UNDER-PROVISIONING

- Performance issues
- Throttling / OOMKills
- Pod instability



Use metrics to find the right balance!

6. MONITORING RESOURCE USAGE

- Monitor actual usage vs requests/limits.
- Use metrics to optimize and set proper values.

kubectl top node

kubectl top pod --all-namespaces

Metrics Server /
Prometheus



Analyze Usage
(CPU / Memory)



Optimize
Requests & Limits



COMMON COMMANDS

```

# View node resources
kubectl top node

# View pod resources
kubectl top pod --all-namespaces

# Describe pod and see QoS class
kubectl describe pod <pod-name> | grep -i qos

# Get resource usage in detail
kubectl get pod <pod-name> -o wide
  
```

RESOURCE MANAGEMENT CHEAT SHEET

CONCEPT	DESCRIPTION
Requests	Minimum resources guaranteed to the container
Limits	Maximum resources the container can use
CPU (m)	1000m = 1 CPU Core
Memory (Mi/Gi)	1024Mi = 1Gi
QoS Guaranteed	requests = limits for CPU & Memory
QoS Burstable	requests < limits or only requests set
QoS BestEffort	No requests, no limits

★ KEY TAKEAWAY

- ✓ Always set requests and limits.
- ✓ Guaranteed Pods are most stable.
- ✓ BestEffort Pods are evicted first.
- ✓ Monitor, analyze, and optimize regularly.
- ✓ Efficient resource usage improves performance and reduces cost.

15. HEALTH CHECKS

“Health checks help Kubernetes understand the state of your containers so it can keep your application available, stable, and self-healing.”

AT A GLANCE

- ☒ Detect and recover from failures
- ☒ Keep traffic away from unready Pods
- ☒ Ensure slow-starting apps have time
- ☒ Configure probes for your app
- ☒ Monitor and troubleshoot probe issues

1. HEALTH CHECKS OVERVIEW

- Kubernetes uses probes to check container health.
- Probes provide signals for decisions like restarts and traffic routing.
- Three types of probes: Liveness, Readiness, Startup.
- Proper configuration is critical for stability.



Kubelet runs probes on containers.
Based on the result, Kubernetes takes action.

2. PROBE TYPES EXPLAINED



LIVENESS PROBE

- Checks if the container is running.
- If it fails, Kubernetes restarts the container.
- Use to detect deadlocks, crashes, or hangs.



READINESS PROBE

- Checks if the container is ready to serve traffic.
- If it fails, Pod is removed from Service endpoints.
- Use for app initialization, dependencies, warm-up.



STARTUP PROBE

- Checks if the container has started successfully.
- Disables liveness & readiness until it succeeds.
- Use for slow-starting applications.

3. PROBE CONFIGURATION OPTIONS

FIELD	DESCRIPTION	EXAMPLE
httpGet	Performs an HTTP GET request	path: /healthz port: 8080
tcpSocket	Performs a TCP socket check	port: 8080
exec	Executes a command in container	command: ["/bin/sh", "-c", "cat /tmp/healthy"]
initialDelaySeconds	Delay before first probe	initialDelaySeconds: 10
periodSeconds	How often to run the probe	periodSeconds: 10
timeoutSeconds	Probe timeout	timeoutSeconds: 3
successThreshold	Consecutive successes needed	successThreshold: 1
failureThreshold	Consecutive failures before action	failureThreshold: 3

4. SAMPLE PROBE CONFIGURATIONS

LIVENESS PROBE (HTTP)

```

livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
  timeoutSeconds: 3
  failureThreshold: 3
  
```

READINESS PROBE (TCP)

```

readinessProbe:
  tcpSocket:
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 3
  
```

STARTUP PROBE (EXEC)

```

startupProbe:
  exec:
    command: ["/bin/sh",  
             "-c", "pgrep -f app"]
  initialDelaySeconds: 0
  periodSeconds: 10
  timeoutSeconds: 3
  failureThreshold: 30
  
```

Startup probe must succeed before liveness and readiness probes start.

5. HOW PROBES WORK (FLOW)



6. TROUBLESHOOTING PROBES

COMMON ISSUES

- Wrong path or port
- App takes too long to start
- Dependencies not ready
- Resource constraints
- Network policies blocking
- Probe timeout too short
- Failure threshold too low

TROUBLESHOOTING STEPS

- Check Pod events: `kubectl describe pod <pod>`
- Check probe logs in container
- Test endpoint inside Pod: `kubectl exec -it <pod> -- curl -v http://localhost:8080/healthz`
- Increase initialDelaySeconds for slow apps
- Verify resources (CPU/Memory)
- Check network policies and firewall rules

COMMON COMMANDS

```

# Describe Pod
kubectl describe pod <pod>

# View Pod events
kubectl get events --sort-by=.metadata.creationTimestamp

# Exec into Pod
kubectl exec -it <pod> -- /bin/sh

# Check endpoints (readiness)
kubectl get endpoints <service> -o wide
  
```

PROBE STATUS QUICK VIEW

PROBE	FAILURE ACTION	AFFECTS TRAFFIC?
Liveness	Restart container	No (directly)
Readiness	Remove from Service endpoints	Yes (stops traffic)
Startup	Keep checking	Yes (until success)



KEY TAKEAWAY

- ☒ Liveness = Is the container alive?
- ☒ Readiness = Is the container ready?
- ☒ Startup = Give the app time to start.
- ☒ Configure probes based on your app behavior.
- ☒ Good probes improve reliability and user experience.

16. SCHEDULING

“

The Kubernetes Scheduler assigns Pods to the most suitable Nodes based on resources, policies, constraints, and rules you define. Good scheduling = high availability, better performance, and efficient resource usage.

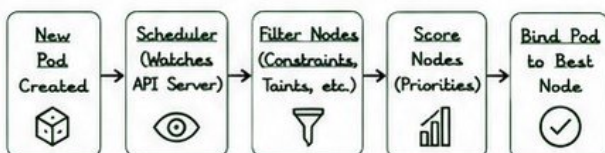
”

AT A GLANCE

- ✓ Scheduler watches for new Pods
- ✓ Evaluates nodes and constraints
- ✓ Applies filters and scoring
- ✓ Respects taints, tolerations, node affinity & anti-affinity
- ✓ Places Pod on the best node

1. SCHEDULER DECISIONS

- Scheduler watches for Pods without a node.
- It evaluates all available nodes.
- Filters out nodes that don't meet requirements.
- Scores the remaining nodes.
- Binds the Pod to the highest scoring node.



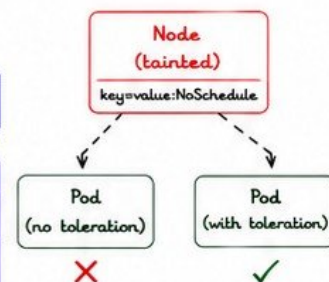
If no node fits, the Pod remains in Pending state until a suitable node becomes available.

2. TAINTS

- Taints repel Pods.
- A taint on a node prevents Pods from being scheduled unless they have a matching toleration.
- Used to reserve nodes for special workloads.

Add a taint to a node
`kubectl taint nodes node1 key=value:NoSchedule`

EFFECTS	DESCRIPTION
NoSchedule	Do not schedule new Pods on the node.
PreferNoSchedule	Try to avoid scheduling new Pods.
NoExecute	Do not schedule new Pods and evict existing Pods that don't tolerate it.



3. TOLERATIONS

- Tolerations allow a Pod to be scheduled on tainted nodes.
- A toleration must match the taint key, value and effect.

tolerations:

- key: "key"
 operator: "Equal"
 value: "value"
 effect: "NoSchedule"

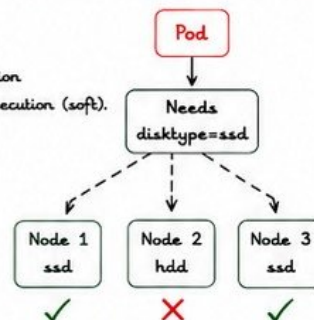
FIELDS	DESCRIPTION
key	Taint key
operator	Equal Exists
value	Taint value (for Equal)
effect	NoSchedule PreferNoSchedule NoExecute

4. NODE AFFINITY

- Attracts Pods to nodes that match rules.
- Types: `requiredDuringSchedulingIgnoredDuringExecution` (hard) and `preferredDuringSchedulingIgnoredDuringExecution` (soft).

affinity:

nodeAffinity:
`requiredDuringSchedulingIgnoredDuringExecution:`
 nodeSelectorTerms:
 - matchExpressions:
 - key: disktype
 operator: In
 values: [ssd]

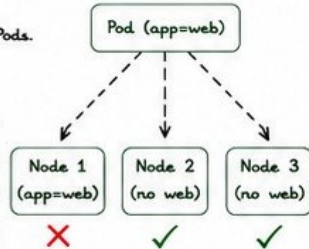


5. ANTI-AFFINITY

- Keeps Pods away from nodes or other Pods.
- Useful for high availability and spreading workloads.

affinity:

podAntiAffinity:
`requiredDuringSchedulingIgnoredDuringExecution:`
 - labelSelector:
 matchLabels:
 app: web
 topologyKey: kubernetes.io/hostname



6. COMMON USE CASES

USE CASE	HOW IT HELPS
Dedicated Nodes	Use taints & tolerations to reserve nodes for critical workloads.
High Availability	Use anti-affinity to spread replicas across different nodes.
Performance	Use affinity to place Pods on nodes with SSD, high CPU, or specific labels.
Cost Optimization	Use affinity to run batch jobs on cheaper or spot nodes.

COMMON COMMANDS

```

# View nodes and taints
kubectl get nodes -o wide

# Describe a node (see taints)
kubectl describe node <node-name>

# Add a taint
kubectl taint nodes <node> key=value:NoSchedule

# Remove a taint
kubectl taint nodes <node> key:NoSchedule-

# View Pod events (scheduling reason)
kubectl describe pod <pod-name>

# Get Pods pending due to scheduling
kubectl get pods --field-selector=status.phase=Pending
  
```

KEY TAKEAWAY

- ✓ Taints repel, tolerations allow.
- ✓ Affinity attracts, anti-affinity spreads.
- ✓ Scheduler makes decisions based on rules and resources.
- ✓ Design scheduling policies for availability, performance and cost efficiency.

17. DAEMONSETS

“

A DaemonSet ensures that all (or selected) Nodes in your cluster run a copy of a Pod. It is ideal for node-level services that must be available everywhere.

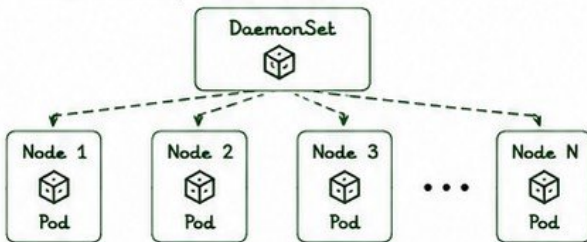
”

AT A GLANCE

- ✓ Run a copy of a Pod on every (or selected) Node.
- ✓ Perfect for node-level agents.
- ✓ Auto-adds Pods to new Nodes.
- ✓ Auto-removes Pods from removed Nodes.
- ✓ Keeps node services consistent.

1. DAEMONSETS OVERVIEW

- Runs one Pod per Node (or per matching Node).
- Great for infrastructure services that run on every node.
- New Nodes automatically get a Pod.
- If a Node is removed, its Pod is garbage collected.
- Ensures consistency across the cluster.



One Pod runs on every Node (unless excluded by rules).

2. WHEN TO USE DAEMONSETS

USE CASE	WHY DAEMONSET?
 Node Agents	Runs agents on every node for management, security, and automation.
 Logging	Collect logs from all nodes and forward to central log storage.
 Monitoring	Collect metrics and node health data from every node.
 Networking	Network plugins, DNS, and proxies that need to run on all nodes.
 Security	Security scanning, auditing, and enforcement at the node level.

3. DAEMONSET YAML EXAMPLE

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-agent
  template:
    metadata:
      labels:
        app: node-agent
    spec:
      containers:
        - name: node-agent
          image: prom/node-exporter:latest
          ports:
            - containerPort: 9100
            name: metrics
  
```

WHAT THIS DOES

- ✓ Runs the node-exporter Pod on every node.
- ✓ Exposes metrics on port 9100.
- ✓ Automatically deploys on new nodes.

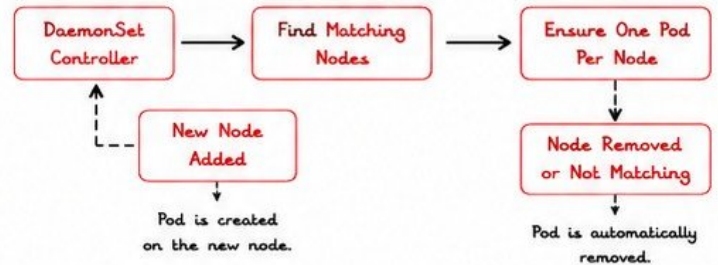
CHECK STATUS

```

kubectl get daemonset -A
kubectl get pods -A
-o wide | grep node-agent
kubectl describe ds node-agent
-n monitoring
  
```

4. HOW DAEMONSETS WORK

- The DaemonSet controller ensures the desired Pod exists on all matching nodes.
- It watches for new nodes and schedules a Pod automatically.
- It removes Pods from nodes that no longer match.



5. ADVANCED OPTIONS

Run only on specific nodes (nodeSelector)

```

spec:
  template:
    spec:
      nodeSelector:
        node-role.kubernetes.io/worker: ""
  
```

Run on specific nodes (nodeAffinity)

```

spec:
  template:
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - key: disktype
              operator: In
              values: [ssd]
  
```

Tolerate taints (run on tainted nodes)

```

spec:
  template:
    spec:
      tolerations:
        - key: "dedicated"
          operator: "Equal"
          value: "monitoring"
          effect: "NoSchedule"
  
```

Update strategy

```

spec:
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
  
```

6. PRODUCTION EXAMPLES

EXAMPLE	TOOL / IMAGE	PURPOSE
Node Exporter	prom/node-exporter	Collect node metrics for Prometheus
Fluent Bit	fluent/fluent-bit	Collect logs from nodes and forward to log storage
Calico / Cilium	calico/node / cilium	Provide networking and network policies
Datadog Agent	datadog/agent	Monitor logs, metrics, and traces on every node
Falco	falcosecurity/falco	Runtime security monitoring on every node

COMMON COMMANDS

```

kubectl get daemonset -A # List all DaemonSets
kubectl describe daemonset <name> -n <ns> # Describe DaemonSet
kubectl get pods -n <ns> -o wide # See Pods on each node
kubectl rollout restart daemonset <name> -n <ns> # Restart Pods
kubectl rollout status daemonset <name> -n <ns> # Check rollout
  
```

KEY TAKEAWAY



- ✓ DaemonSets run a copy of a Pod on every (or selected) node.
- ✓ Ideal for node-level services: monitoring, logging, networking, security, and agents.
- ✓ They automatically adapt to new nodes and changes.
- ✓ Use nodeSelector, affinity, tolerations, and taints for control.

BEST PRACTICES

- ✓ Keep DaemonSet Pods lightweight.
- ✓ Use resource requests and limits.
- ✓ Add liveness/readiness probes.
- ✓ Use tolerations for tainted nodes.
- ✓ Monitor DaemonSet health regularly.

18. STATEFULSETS

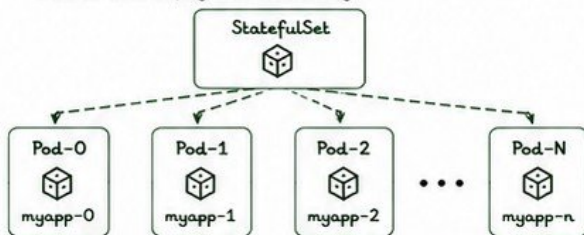
AT A GLANCE

- ✓ Stable network identities
- ✓ Ordered deployment & scaling
- ✓ Persistent storage per Pod
- ✓ Built-in scaling and updates
- ✓ Perfect for stateful workloads

“ StatefulSets manage stateful applications. They provide stable identities, ordered operations, and persistent storage for each Pod. ”

1. STATEFULSETS OVERVIEW

- Designed for stateful applications.
- Each Pod gets a stable hostname and identity.
- Provides stable storage using PersistentVolumeClaims.
- Supports ordered deployment, scaling, and updates.
- Ensures data safety and consistency.



Each Pod has a stable network identity and its own persistent storage.

2. KEY FEATURES

FEATURE	DESCRIPTION
Stable Identities	Each Pod gets a predictable hostname (e.g., myapp-0, myapp-1).
Ordered Deployment	Pods are created in order (0 → 1 → 2 ...) and started one at a time.
Persistent Storage	Each Pod gets its own PersistentVolumeClaim that survives restarts.
Scaling	Scale up/down with ordered operations to maintain consistency.
Stateful Workloads	Ideal for databases, message brokers, and distributed systems.

3. STATEFULSET YAML EXAMPLE

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: webapp
spec:
  serviceName: "webapp"
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - name: web
          image: nginx
          ports:
            - containerPort: 80
  volumeClaimTemplates:
    - metadata:
        name: web-data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 10Gi
  
```

WHAT THIS DOES

- ✓ Creates 3 Pods: webapp-0, webapp-1, webapp-2
- ✓ Each Pod gets its own PVC (web-data-0, web-data-1, web-data-2)
- ✓ Ensures stable hostnames and persistent storage

POD HOSTNAMES

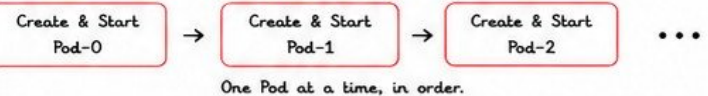
webapp-0.webapp.default.svc.cluster.local
webapp-1.webapp.default.svc.cluster.local
webapp-2.webapp.default.svc.cluster.local

PVCs CREATED

web-data-0 → for webapp-0
web-data-1 → for webapp-1
web-data-2 → for webapp-2

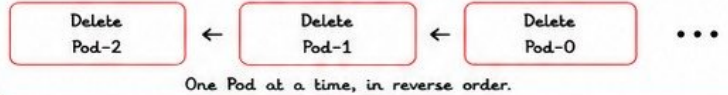
4. ORDERED OPERATIONS

DEPLOYMENT / SCALING UP



One Pod at a time, in order.

SCALING DOWN / DELETION



One Pod at a time, in reverse order.

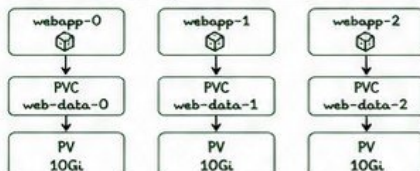
ROLLING UPDATE



Maintains availability and data consistency.

5. PERSISTENT STORAGE PER POD

- Each Pod gets a dedicated PersistentVolumeClaim.
- Storage is not shared between Pods.
- Data remains even if the Pod is deleted and recreated.



★ IMPORTANT

Do NOT delete PVCs when deleting Pods unless you want to lose the data.

6. STATEFUL WORKLOAD EXAMPLES

WORKLOAD	WHY STATEFULSET?
Databases (MySQL, PostgreSQL)	Need stable identities and persistent storage.
Message Brokers (Kafka, RabbitMQ)	Require ordered startup and stable networking.
Search Engines (Elasticsearch)	Data must persist and nodes must be stable.
Distributed Systems (Zookeeper, etcd)	Require consistent quorum and stable nodes.

COMMON COMMANDS

```

kubectl get statefulsets
kubectl get pods -l app=webapp -o wide
kubectl describe statefulset webapp
kubectl scale statefulset webapp --replicas=5
kubectl delete statefulset webapp
  
```

BEST PRACTICES

- ✓ Use StatefulSet for stateful applications only.
- ✓ Always use PersistentVolumeClaims.
- ✓ Keep replicas ≥ 3 for high availability.
- ✓ Monitor storage and Pod health.
- ✓ Test scaling and recovery regularly.



KEY TAKEAWAY

StatefulSets provide stable identities, ordered operations, and persistent storage, making them ideal for databases, message brokers, and any stateful workload.

19. JOBS AND CRONJOBS

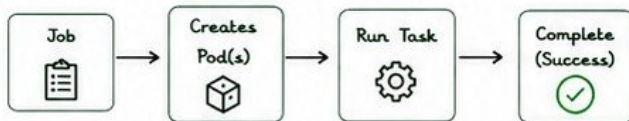
“Jobs run tasks to completion. CronJobs run Jobs on a schedule. They are ideal for batch processing, maintenance, backup, report generation, and data pipelines.”

AT A GLANCE

- ✓ Run one-time or scheduled tasks
- ✓ Perfect for batch processing
- ✓ Handle parallel jobs easily
- ✓ Automatic cleanup of completed jobs
- ✓ Monitor job success and failures

1. JOBS OVERVIEW

- Jobs create one or more Pods and run until completion.
- Ideal for batch jobs, migrations, backups, data processing.
- Supports retries on failure.
- Can run multiple Pods in parallel.



When the task completes successfully (or retries are exhausted), the Job is marked Complete (or Failed).

2. JOB TYPES

TYPE	DESCRIPTION
Non-Parallel Job	Runs a single Pod until completion. (completions: 1, parallelism: 1)
Parallel Job	Runs multiple Pods in parallel to complete the work faster (parallelism > 1)
Indexed Job	Each Pod gets a unique index (0..N-1). Useful for sharded or partitioned work.
CronJob	Runs Jobs on a schedule (like cron in Linux). Creates a new Job at each schedule time.

3. JOB YAML EXAMPLE

```

apiVersion: batch/v1
kind: Job
metadata:
  name: batch-job
spec:
  completions: 1 # Total successful completions required
  parallelism: 3 # Number of Pods to run in parallel
  backoffLimit: 4 # Retry limit before Job fails
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: worker
          image: busybox
          command: ["sh", "-c", "echo Processing...; sleep 10"]
  
```

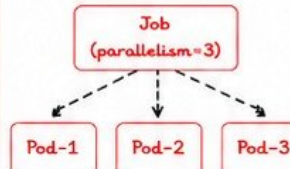
KEY FIELDS

- completions:**
Total successful Pods required.
- parallelism:**
Pods running at once.
- backoffLimit:**
Retries on failure.

4. PARALLEL & INDEXED JOBS

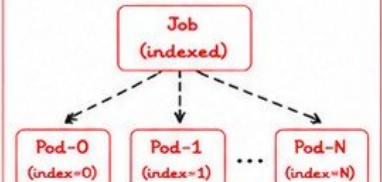
PARALLEL JOB

- Uses parallelism > 1
- All Pods share the same workload.
- Good for splitting large tasks.



INDEXED JOB

- Uses completionMode: Indexed
- Each Pod gets a unique index.
- Ideal for sharded or partitioned processing.



5. CRONJOB YAML EXAMPLE

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-report
spec:
  schedule: "0 2 * * *" # At 02:00 AM daily
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 2
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: alpine
              command: ["sh", "-c", "echo Generating report..."]
  
```

CRON SCHEDULE FORMAT

```

* * * * *
| | | | |
| | | +--- Day of week (0-6)
| | | +--- Month (1-12)
| | +----- Day of month (1-31)
| +----- Hour (0-23)
+----- Minute (0-59)
  
```

EXAMPLES

```

0 2 * * * Every day at 02:00 AM
*/15 * * * * Every 15 minutes
0 0 * * 0 Every Sunday at 00:00
  
```

6. JOBS & CRONJOBS BEST PRACTICES

	Idempotent Tasks	Design jobs to be idempotent so re-runs do not cause issues.
	Use Backoff Limit	Set appropriate backoffLimit to handle transient failures.
	Limit Concurrency	Control parallelism to avoid overloading systems.
	Cleanup Completed Jobs	Use ttlSecondsAfterFinished to automatically clean up finished Jobs.
	Monitor Jobs	Track job status, failures, and durations using Prometheus/Grafana or logs.

COMMON COMMANDS

```

kubectl create job my-job --image=busybox # Create a job
kubectl get jobs # List all jobs
kubectl describe job my-job # Describe a job
kubectl logs job/my-job # View job logs
kubectl delete job my-job # Delete a job
kubectl get cronjobs # List all cronjobs
kubectl describe cronjob my-cron # Describe a cronjob
kubectl create -f cronjob.yaml # Create cronjob
kubectl delete cronjob my-cron # Delete cronjob
  
```

JOB STATUS PHASES

PHASE	DESCRIPTION
Active	Job is running.
Complete	Job completed successfully.
Failed	Job failed (retries exhausted).
BackoffLimitExceeded	Job failed after reaching retry limit.
Suspended	CronJob is suspended.

KEY TAKEAWAY

- ✓ Jobs run tasks to completion.
- ✓ CronJobs run Jobs on a schedule.
- ✓ Use parallelism and indexed Jobs for large workloads.
- ✓ Clean up and monitor Jobs for reliability and efficiency.



kubernetes

VERIQTA

PAGE 20

20. KUBERNETES NETWORKING

“

Kubernetes networking allows Pods, Services, and external clients to communicate reliably and securely across the cluster and beyond. Understanding the networking model is essential for debugging and designing production systems.

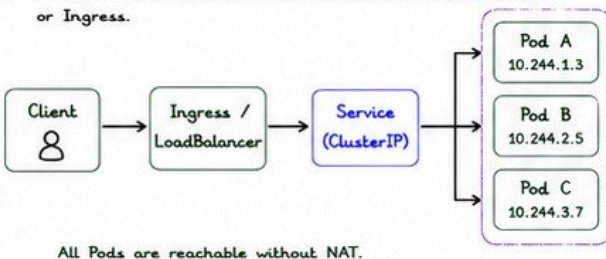
”

AT A GLANCE

- ✓ CNI provides Pod networking
- ✓ Pods get IPs and communicate across nodes
- ✓ Services provide stable access to Pods
- ✓ DNS resolves Services and Pods
- ✓ Understand traffic flow end-to-end

1. KUBERNETES NETWORKING OVERVIEW

- Every Pod gets its own IP address.
- Pods can communicate with each other across all nodes.
- Services provide a stable virtual IP and DNS name.
- Traffic flows through kube-proxy using iptables/IPVS.
- External clients access Services via NodePort, LoadBalancer or Ingress.



2. CNI (CONTAINER NETWORK INTERFACE)

- CNI plugins configure network interfaces in Pods.
- They provide IPAM (IP management) and routing.
- Popular CNI plugins: Calico, Cilium, Flannel, Weave Net.

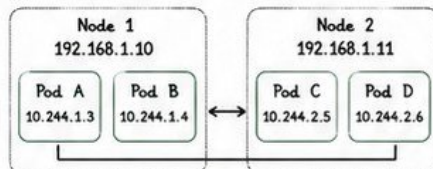
CNI PLUGIN	FEATURES
Calico	Policy enforcement, BGP routing, NetworkPolicies
Cilium	eBPF-based, high performance, L7 policies, Observability
Flannel	Simple overlay network, easy to deploy
Weave Net	Simple, encrypted overlay, built-in DNS

CHECK CNI STATUS
 kubectl get pods -n kube-system
 kubectl logs -n kube-system <cni-pod>

TIP
 CNI must be healthy before any Pods can communicate.

3. POD NETWORKING

- Pods communicate using their IP addresses.
- Communication across nodes is handled by the CNI plugin.
- All Pods are in a flat network by default.



CHECK POD NETWORKING
 kubectl get pods -o wide
 kubectl exec -it <pod-a> -- ping <pod-b-ip>

KEY POINTS

- ✓ No NAT between Pods.
- ✓ Pods can reach Pods on other nodes.
- ✓ NetworkPolicies control allowed traffic.

4. SERVICE NETWORKING

- Services provide a stable IP and DNS name for Pods.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.
- kube-proxy programs iptables or IPVS to route traffic.

SERVICE TYPE	ACCESS	USE CASE
ClusterIP	Internal (default)	Exposes Service inside cluster
NodePort	<NodeIP>:<NodePort>	Expose Service on each node IP
LoadBalancer	Cloud LB IP	Expose Service via cloud provider
ExternalName	DNS CNAME	Map Service to external DNS name

CHECK SERVICES
 kubectl get svc -o wide
 kubectl describe svc <service-name>

NOTE
 ClusterIP is default and only reachable inside the cluster.

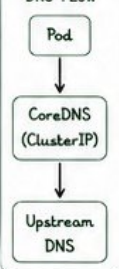
5. DNS IN KUBERNETES

- CoreDNS (or kube-dns) provides DNS for Services and Pods.
- Every Service gets a DNS name: <service>.<namespace>.svc.cluster.local
- Pods can resolve Services and other Pods by name.

WHAT CAN BE RESOLVED?	EXAMPLE
Service (same namespace)	my-svc.default.svc.cluster.local
Service (other namespace)	my-svc.prod.svc.cluster.local
Pod (same namespace)	pod-ip or pod-name (if headless svc)
External domain	google.com

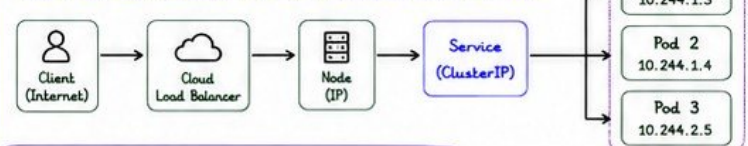
TEST DNS
 kubectl exec -it <pod> -- nslookup my-svc
 kubectl exec -it <pod> -- dig my-svc.default.svc.cluster.local

DNS FLOW



6. TRAFFIC FLOW EXAMPLE

EXAMPLE: CLIENT ACCESSING A POD VIA LOADBALANCER SERVICE



1. Client sends traffic to the Load Balancer.
2. LB forwards traffic to a Node IP.
3. kube-proxy forwards traffic to the Service (ClusterIP).
4. Service load balances traffic to one of the healthy Pods.
5. Response travels back the same path to the client.

KEY COMPONENTS

- CNI: Pod IP & routing
- kube-proxy: Service routing
- CoreDNS: Name resolution
- Ingress/LB: External access

COMMON COMMANDS

kubectl get nodes -o wide # View nodes and IPs
 kubectl get pods -o wide # View pods and IPs
 kubectl get svc -o wide # View services and IPs
 kubectl get endpoints <svc> # View endpoint IPs
 kubectl describe pod <pod> # Pod networking details

TROUBLESHOOTING CHECKLIST

- ✓ CNI pods running?
- ✓ Do Pods have IP addresses?
- ✓ Can Pods ping each other?
- ✓ Does CoreDNS resolve Services?
- ✓ Are Services and Endpoints correct?
- ✓ Are NetworkPolicies blocking traffic?



KEY TAKEAWAY

- ✓ Pods communicate directly across nodes.
- ✓ Services provide stable access and load balancing.
- ✓ DNS makes discovery easy.
- ✓ Understand traffic flow to debug issues fast.
- ✓ Networking is the foundation of Kubernetes reliability.

VERIQTA

veriqta

veriqta

veriqta

@veriqta

@veriqta

@veriqta

Learn. Practice. Build. Deploy. Repeat.



kubernetes

VERIQTA

21. COREDNS

PAGE 21

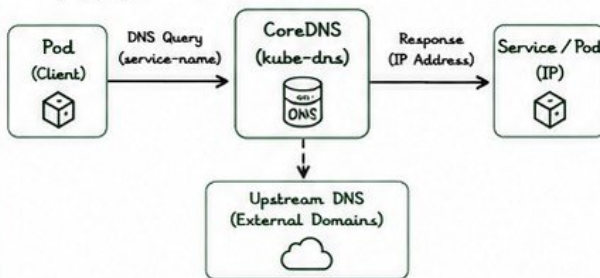
AT A GLANCE

- ✓ CoreDNS is the DNS server in Kubernetes.
- ✓ Provides service discovery and DNS resolution.
- ✓ Resolves Services, Pods, and external domains.
- ✓ Highly configurable and extensible.
- ✓ Tune for performance, reliability, and scale.

CoreDNS provides DNS resolution and service discovery in Kubernetes. It allows Pods and Services to find each other by name and communicate reliably across the cluster and with external systems.

1. COREDNS OVERVIEW

- Default DNS server in Kubernetes.
- Runs as a Deployment in the kube-system namespace.
- Answers DNS queries from Pods and Services.
- Uses ConfigMap for configuration.
- Highly plugin-based and extensible.



2. SERVICE DISCOVERY

- CoreDNS enables discovery of Services and Pods.

QUERY	RESOLVES TO
my-service.default.svc.cluster.local	ClusterIP of the Service
my-service.default.svc	ClusterIP of the Service
my-pod.my-namespace.pod.cluster.local	Pod IP
my-pod.my-namespace.pod	Pod IP
*.default.svc.cluster.local	All Services in default namespace
kubernetes.default.svc.cluster.local	Kubernetes API Server IP

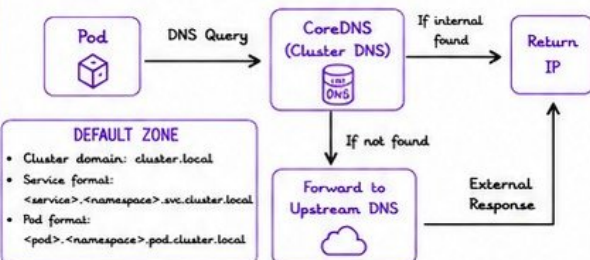
```

$ # Test service resolution
kubectl run dnstools --image=busybox --rm -it --restart=Never -- nslookup my-service

$ # Test external domain
kubectl run dnstools --image=busybox --rm -it --restart=Never -- nslookup google.com
  
```

3. DNS RESOLUTION FLOW

- Pods send DNS queries to CoreDNS.
- CoreDNS checks internal zone (cluster.local) first.
- If not found, forwards query to upstream DNS.



DEFAULT ZONE

- Cluster domain: cluster.local
- Service format: <service>.<namespace>.svc.cluster.local
- Pod format: <pod>.<namespace>.pod.cluster.local

4. DNS TROUBLESHOOTING

- DNS issues are common and can break applications.
- Use these steps to troubleshoot.

STEP	CHECK	COMMAND / ACTION
1	CoreDNS Pods Running	<code>kubectl get pods -n kube-system -l k8s-app=kube-dns</code>
2	Pod Can Reach CoreDNS	<code>kubectl run dnstools -it --rm --image=busybox -- nslookup kubernetes.default</code>
3	Service Exists	<code>kubectl get svc <service-name> -n <namespace></code>
4	Endpoints Exist	<code>kubectl get endpoints <service-name> -n <namespace></code>
5	CoreDNS Logs	<code>kubectl logs -n kube-system -l k8s-app=kube-dns</code>
6	ConfigMap Valid	<code>kubectl -n kube-system get configmap coredns -o yaml</code>

5. COREDNS CONFIGURATION

- CoreDNS is configured using a ConfigMap.
- Default ConfigMap: coredns in kube-system namespace.
- Key plugins and their order matter.

EXAMPLE COREFILE	PLUGIN	PURPOSE
<pre> :53 { errors health { lameduck 5s } ready kubernetes cluster.local in-addr.arpa ip6.arpa { pods insecure fallthrough in-addr.arpa ip6.arpa } forward . /etc/resolv.conf cache 30 loop reload loadbalance } </pre>	errors	Logs errors
	health	Health check endpoint
	ready	Readiness probe
	kubernetes	Service & Pod discovery
	forward	Forward to upstream DNS
	cache	Cache DNS responses
	reload	Reload config on change
	loadbalance	Balance queries across upstreams

6. PRODUCTION TUNING

- Optimize CoreDNS for performance, reliability, and scale.

TUNING AREA	RECOMMENDATION
Replicas	Run 2+ replicas for high availability.
Resources	Set requests/limits (CPU & Memory) appropriately.
Cache	Use cache plugin (e.g., cache 30) to reduce latency.
Upstream DNS	Configure reliable upstream DNS servers.
Node Local DNS	Use NodeLocal DNSCache for high QPS clusters.
Monitoring	Monitor CoreDNS metrics and logs.
Logging	Enable log plugin for debugging (remove in prod).

BEST PRACTICES

- ✓ Run multiple CoreDNS replicas.
- ✓ Set resource requests & limits.
- ✓ Enable caching for performance.
- ✓ Use reliable upstream DNS.
- ✓ Monitor CoreDNS health & metrics.
- ✓ Limit excessive logging in production.
- ✓ Test DNS regularly.

COMMON COMMANDS

```

kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl -n kube-system get svc kube-dns
kubectl -n kube-system get configmap coredns -o yaml
kubectl logs -n kube-system -l k8s-app=kube-dns
kubectl exec -it <pod> -- cat /etc/resolv.conf
kubectl run dnstools --image=busybox --rm -it --restart=Never --
-- nslookup <service>
kubectl run dnstools --image=busybox --rm -it --restart=Never --
-- nslookup <domain>
  
```

COREDNS METRICS (EXAMPLES)

```

coredns_dns_requests_total
Total DNS requests
coredns_dns_responses_total
Total DNS responses
coredns_dns_request_duration_seconds
DNS request latency
coredns_health_request_duration_seconds
Health check latency
  
```



KEY TAKEAWAY

- ✓ CoreDNS is critical for service discovery and DNS.
- ✓ Understand how DNS resolution works in Kubernetes.
- ✓ Use proper configuration and tuning for production.
- ✓ Monitor and troubleshoot DNS issues quickly.
- ✓ A healthy DNS = a healthy cluster.

22. NETWORKPOLICIES

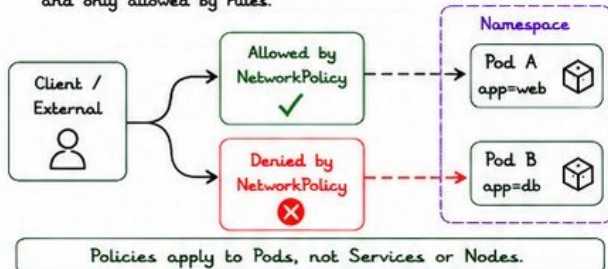
“NetworkPolicies provide a way to control the traffic flow to and from Pods based on rules. They enforce isolation and allow only the traffic you explicitly permit.”

AT A GLANCE

- ✓ NetworkPolicies control traffic to and from Pods.
- ✓ Pods are isolated by default (deny all) when a policy selects them.
- ✓ Define Ingress and Egress rules using selectors and ports.
- ✓ Essential for security, compliance, and multi-tenant isolation.

1. NETWORKPOLICIES OVERVIEW

- Applied at the namespace level.
- Selects Pods using label selectors.
- Defines Ingress (incoming) and Egress (outgoing) rules.
- If no policy selects a Pod → all traffic is allowed.
- If a policy selects a Pod → traffic is denied by default and only allowed by rules.



2. CORE CONCEPTS

CONCEPT	DESCRIPTION
 Pod Selection	Use labelSelector to select Pods to apply the policy.
 Ingress Rules	Allow incoming traffic to selected Pods from specific sources and ports.
 Egress Rules	Allow outgoing traffic from selected Pods to specific destinations and ports.
 Default Deny	Once a policy selects a Pod, all traffic is denied unless explicitly allowed.
 Additive	Multiple policies are additive (union of all allow rules).

3. NETWORKPOLICY YAML EXAMPLE (INGRESS)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-from-frontend
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 80
```

WHAT THIS DOES

- ✓ Selects Pods with label app=web.
- ✓ Allows traffic to port 80 only.
- ✓ Only from Pods with label role=frontend.

RESULT

Frontend Pods can reach Web Pods on port 80.
All other traffic to Web Pods is denied.

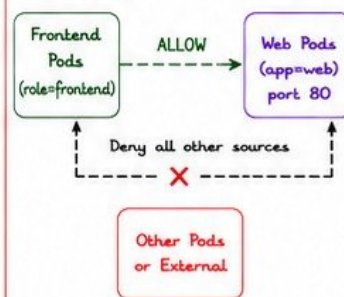
NOTE

If egress rules are not defined, outgoing traffic is allowed by default (unless an egress policy exists).

4. INGRESS AND EGRESS RULES

INGRESS RULES

Allow traffic INTO selected Pods.



EGRESS RULES

Allow traffic FROM selected Pods.



5. EXAMPLE: EGRESS POLICY

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api-egress-db
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Egress
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: db
      ports:
        - protocol: TCP
          port: 5432
```

WHAT THIS DOES

- ✓ Selects Pods with label app=api.
- ✓ Allows egress traffic only to Pods with label app=db on port 5432.
- ✓ Blocks all other destinations.

IMPORTANT

To enforce egress restrictions, at least one egress policy must exist for the Pods. Otherwise, egress is allowed by default.

6. PRODUCTION DESIGN PRINCIPLES

 Default Deny	Start with deny all, then allow what you need.
 Least Privilege	Allow minimal required ports and sources.
 Use Labels	Use stable, meaningful labels for selection.
 Namespace Isolation	Apply policies per namespace for tenant isolation.
 Document Rules	Document purpose of each policy and rule.
 Test Thoroughly	Test connectivity after applying policies.

COMMON COMMANDS

```
# List NetworkPolicies
kubectl get networkpolicies -A
# Describe a policy
kubectl describe networkpolicy <name> -n <ns>
# Get Pods with labels (for selector check)
kubectl get pods --show-labels -n <ns>
# Test connectivity from Pod
kubectl exec -it <pod> -- curl -I http://<ip>:80
```

CHECKLIST

- ✓ Identify Pods and labels clearly.
- ✓ Define ingress and egress requirements.
- ✓ Start with default deny.
- ✓ Allow only required ports and sources.
- ✓ Test connectivity in all directions.
- ✓ Monitor and audit traffic regularly.



KEY TAKEAWAY

- ✓ NetworkPolicies protect your Pods.
- ✓ Once a policy selects a Pod, deny all unless allowed.
- ✓ Use ingress and egress rules carefully.
- ✓ Design for security, compliance, and isolation.
- ✓ Test, monitor, and refine your policies.

23. HORIZONTAL POD AUTOSCALER

66

The Horizontal Pod Autoscaler automatically scales the number of Pods in a Deployment or ReplicaSet based on observed metrics. It ensures your application can handle load while optimizing resource usage.

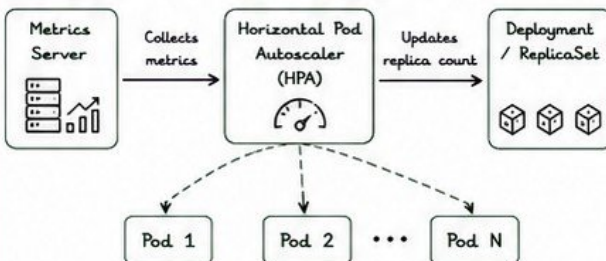
99

AT A GLANCE

- ✓ HPA scales Pods based on metrics.
- ✓ Metrics Server provides CPU/Memory.
- ✓ Supports CPU, Memory, and Custom metrics.
- ✓ Scaling policies control behavior.
- ✓ Keeps applications available and efficient.

1. HPA OVERVIEW

- HPA changes the replica count of a Deployment/ReplicaSet.
- It uses metrics to decide when to scale up or down.
- Requires Metrics Server for resource metrics.
- Works in the same namespace as the target workload.



HPA adjusts the number of Pods to match the demand.

2. HOW HPA WORKS

1. Metrics Server collects CPU and Memory usage from Pods.
2. HPA compares current metrics with the target values.
3. If usage > target → scale up (within maxReplicas).
4. If usage < target → scale down (within minReplicas).
5. HPA updates the Deployment/ReplicaSet replica count.

COMPONENT	ROLE
Metrics Server	Collects resource metrics from kubelets.
HPA Controller	Calculates desired replicas and updates workload.
Target Workload	Deployment or ReplicaSet to be scaled.
kube-apiserver	Stores HPA object and current metrics.



TIP Install Metrics Server before using HPA.

kubectl apply -f <https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml>

3. HPA YAML EXAMPLE (CPU BASED)

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
  
```

WHAT THIS DOES

- ✓ Targets Deployment "webapp".
- ✓ Keeps at least 2 Pods.
- ✓ Scales up to 10 Pods.
- ✓ Maintains average CPU utilization at 60%.

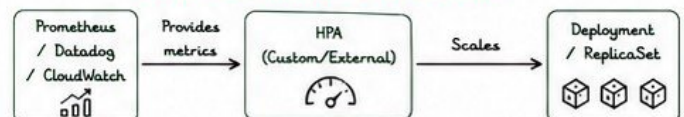
RESULT EXAMPLE

If CPU > 60% → scale up.
If CPU < 60% → scale down.
Always stays between 2 and 10 Pods.

4. METRIC TYPES

METRIC TYPE	DESCRIPTION	EXAMPLE
CPU Utilization	Percentage of CPU used.	Average CPU > 60%
Memory Utilization	Percentage of Memory used.	Average Memory > 70%
Custom Metrics	Application or business metrics.	Requests per second
External Metrics	Metrics from external systems.	Queue length in AWS SQS

CUSTOM & EXTERNAL METRICS EXAMPLE



Requires Metrics Adapter (e.g., prometheus-adapter)

5. SCALING POLICIES (BEHAVIOR)

- Control how fast or slow scaling happens.
- Available in autoscaling/v2 API.
- Configure separate rules for scaleUp and scaleDown.

SCALE UP EXAMPLE	SCALE DOWN EXAMPLE
<pre> type: Percent value: 100 periodSeconds: 60 meaning: can scale up 100% of current Pods every 60s </pre>	<pre> type: Pods value: 2 periodSeconds: 60 meaning: can scale down by 2 Pods every 60s </pre>
<pre> behavior: scaleUp: policies: - type: Percent value: 100 periodSeconds: 60 selectPolicy: Max stabilizationWindowSeconds: 300 # wait 5 minutes before scaling down </pre>	<pre> behavior: scaleDown: policies: - type: Pods value: 2 periodSeconds: 60 selectPolicy: Min </pre>

6. HPA STATUS & COMMANDS

CHECK HPA STATUS

kubectl get hpa

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
webapp-hpa	Deployment/webapp	cpu: 65%/60%	2	10	5	3m

DESCRIBE HPA

kubectl describe hpa webapp-hpa

USEFUL COMMANDS

- # Get HPA
kubectl get hpa
- # Describe HPA
kubectl describe hpa <name>
- # Edit HPA
kubectl edit hpa <name>
- # Delete HPA
kubectl delete hpa <name>
- # Top Pods (requires metrics-server)
kubectl top pods
- # Top Nodes
kubectl top nodes

COMMON ISSUES

- Metrics Server not installed → no metrics.
- HPA shows <unknown>/N/A in TARGETS.
- Insufficient permissions (RBAC).
- Wrong metric name in custom metrics.
- Workload type not supported (e.g., DaemonSet).

TROUBLESHOOTING CHECKLIST

- ✓ Metrics Server running (kubectl top nodes).
- ✓ HPA events (kubectl describe hpa <name>).
- ✓ Metrics visible and valid.
- ✓ Target workload healthy.
- ✓ Correct metric names and adapters.



KEY TAKEAWAY

- ✓ HPA keeps your application responsive.
- ✓ Use the right metrics and targets.
- ✓ Configure scaling policies to avoid flapping.
- ✓ Monitor and tune for your workload.
- ✓ Test scaling under real load.

24. CLUSTER AUTOSCALER

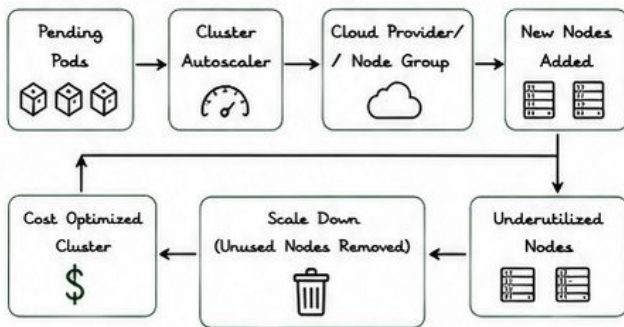
“ The Cluster Autoscaler automatically adjusts the number of nodes in your cluster based on pending Pods. It ensures optimal capacity, high availability, and cost efficiency. ”

AT A GLANCE

- ✓ Scales nodes up when Pods cannot be scheduled.
- ✓ Scales nodes down when utilization is low.
- ✓ Works with Node Groups / ASGs.
- ✓ Understands scheduling constraints.
- ✓ Helps reduce cost and improve efficiency.

1. CLUSTER AUTOSCALER OVERVIEW

- Monitors for unschedulable Pods.
- Determines if more nodes are needed.
- Adds nodes via Cloud Provider / Node Group.
- Removes underutilized nodes safely.
- Respects scheduling constraints and policies.



2. HOW IT WORKS

- A Pod is created but remains Pending (unschedulable).
- CA analyses why it is pending.
- Determine the smallest instance that can fit the Pod.
- Adds a node to the appropriate Node Group.
- Pod is scheduled and runs.
- When nodes are underutilized, CA scales them down.

COMPONENT	ROLE
Cluster Autoscaler	Core component that makes scaling decisions.
Cloud Provider	Adds or removes nodes in the infrastructure.
Node Groups (ASG)	Groups of nodes with same instance type.
Kubernetes Scheduler	Places Pods on the new nodes.
Metrics Source	Uses resource requests to make decisions.



TIP The Cluster Autoscaler does NOT consider actual CPU usage, it uses resource requests and binpacking simulation.

3. NODE GROUPS

- CA works with Node Groups (Auto Scaling Groups).
- Each group has a min, max, and desired size.
- CA scales nodes only within configured limits.

SETTING	DESCRIPTION
Min Size	Minimum number of nodes in the group.
Max Size	Maximum number of nodes in the group.
Desired Size	Initial number of nodes.
Instance Type	Type of nodes in the group.

EXAMPLE (AWS ASG)

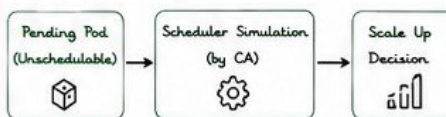
Min Size: 2
Desired Size: 2
Max Size: 10
Instance Type: m5.large

BEST PRACTICE

- ✓ Use multiple Node Groups for different workloads.
- ✓ Use spot & on-demand mix for cost savings.

4. SCHEDULING INTERACTION

- CA simulates scheduling using the Kubernetes Scheduler.
- Considers:
 - ✓ Resource Requests (CPU, Memory).
 - ✓ Taints & Tolerations.
 - ✓ Node Affinity / Anti-affinity.
 - ✓ Pod Disruption Budgets.
 - ✓ Topology Constraints.



NOTE

If Pod cannot be scheduled even after scale up, check scheduling constraints, quotas, or misconfigurations.

5. COST OPTIMIZATION

- Remove underutilized nodes automatically.
- Use right-sized instances for workloads.
- Combine On-Demand + Spot instances.
- Check for idle nodes and scale down.
- Use expander strategies wisely.

EXPANDER STRATEGIES

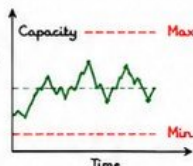
STRATEGY	DESCRIPTION
random	Randomly selects a node group.
least-waste	Minimizes unused resources.
most-pods	Packs more pods per node.
priority	Uses node group priority.



If use multiple node groups and priorities to control workload placement and cost.

6. CAPACITY PLANNING

- Analyse workload trends.
- Define min & max limits carefully.
- Keep buffer for spikes.
- Use multiple instance types.
- Review and test scaling behavior.



7. COMMON COMMANDS

- Check Cluster Autoscaler logs
`kubectl -n kube-system logs deployment/cluster-autoscaler`
- Describe CA
`kubectl -n kube-system describe deployment cluster-autoscaler`
- Check Node Group (AWS example)
`aws autoscaling describe-auto-scaling-groups`
- View nodes
`kubectl get nodes -o wide`

KEY TAKEAWAY



- ✓ Cluster Autoscaler = Right size + Right time.
- ✓ Eliminates manual scaling.
- ✓ Ensures high availability.
- ✓ Reduces cost and overprovisioning.
- ✓ Plan capacity and set limits wisely.

CHECKLIST



- ✓ Cluster Autoscaler running and healthy.
- ✓ Node Groups configured with correct limits.
- ✓ Metrics Server installed.
- ✓ Pods have resource requests.
- ✓ CA logs show no errors.
- ✓ Scale up and scale down tested.



TROUBLESHOOTING

- ✓ Pods still Pending? Check events and constraints.
- ✓ Nodes not scaling up? Verify CA logs and node groups.
- ✓ Scale down not happening? Check utilization and PDBs.
- ✓ Wrong instance type? Review node group settings.
- ✓ CA not running? Check deployment and RBAC.



BEST PRACTICES

- ✓ Use multiple Node Groups for flexibility.
- ✓ Set appropriate min/max limits.
- ✓ Monitor costs and idle nodes.
- ✓ Use mixed instances (Spot + On-Demand).
- ✓ Test scaling regularly.



kubernetes

VERIQTA

25. MONITORING

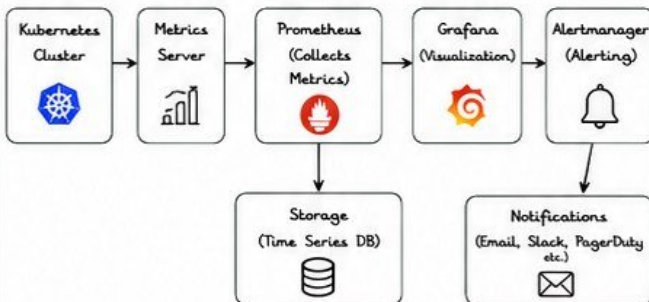
PAGE 25

AT A GLANCE

- ✓ Collect metrics, logs, and events.
- ✓ Visualize and alert in real time.
- ✓ Understand system behavior.
- ✓ Detect issues early.
- ✓ Improve reliability and performance.

“ Monitoring tells you what is happening.
Observability helps you understand why.
Together they help you act before users feel it. ”

1. MONITORING ARCHITECTURE OVERVIEW



★ Metrics flow from cluster → Prometheus → Grafana → Alerts when needed.

2. COMPONENTS EXPLAINED

COMPONENT	ROLE
Metrics Server	Provides CPU and Memory metrics to Kubernetes. Used by HPA and kubectl top.
Prometheus	Scrapes and stores metrics from targets (Pods, Nodes, Services, kube-state-metrics, etc.).
Grafana	Visualizes metrics from Prometheus with dashboards.
Alertmanager	Handles alerts from Prometheus and routes them to the right receivers.
Observability	Unified view of metrics, logs, traces, and events to understand system behavior.

💡 TIP: Monitoring = Knowing WHAT is happening.
Observability = Understanding WHY it is happening.

3. METRICS SOURCES

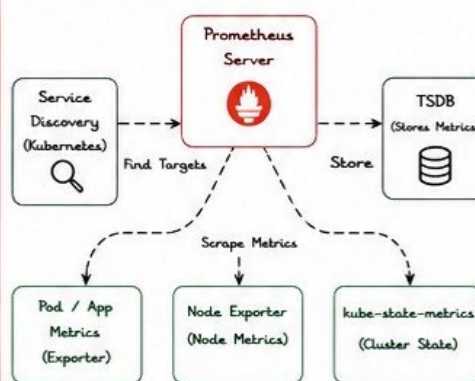
- Kubernetes API (kube-state-metrics)
- Nodes (cAdvisor, node-exporter)
- Applications (instrumentation / exporters)
- Infrastructure (cloud metrics, network, storage)
- Custom business metrics



Common Metrics

Category	Examples
Cluster	Nodes, Pods, Deployments, Services
Resources	CPU, Memory, Disk, Network
Application	Requests, Errors, Latency, Throughput
Business	Transactions, Signups, Orders

4. PROMETHEUS SCRAPE FLOW



5. GRAFANA DASHBOARDS

- Real-time visualization of metrics.
- Pre-built dashboards for Kubernetes.
- Custom dashboards for applications.
- Drill down from cluster → node → pod.
- Share dashboards with teams.

Popular Dashboards

Dashboard	Purpose
Kubernetes Cluster	Overall cluster health
Kubernetes Nodes	Node performance
Kubernetes Pods	Pod performance
Kubernetes / API	API server health
Application	App-specific metrics

★ TIP: Keep dashboards simple, actionable, and focused on SLOs.

6. ALERTMANAGER WORKFLOW



Alert Routing

Route	Purpose
Severity: Critical	PagerDuty + Slack + Email
Severity: Warning	Slack + Email
Team: DevOps	devops-channel
Team: Platform	platform-channel

Alert Best Practices

- ✓ Alert on symptoms, not root causes.
- ✓ Avoid alert storms (use grouping).
- ✓ Set meaningful thresholds.
- ✓ Test alerts regularly.
- ✓ Silence during maintenance windows.

7. OBSERVABILITY PILLARS

Metrics (What)

- Quantitative data
- CPU, Memory, Errors
- Time series data
- Good for trends and alerting

Logs (What Happened)

- Discrete events
- Debug and audit
- Searchable text
- Good for root cause analysis

Traces (Why / How)

- Request journey
- Distributed systems
- Latency breakdown
- Good for performance debugging

Full observability = Metrics + Logs + Traces + Events

8. PRODUCTION MONITORING CHECKLIST

- ✓ Metrics Server installed and healthy
- ✓ Prometheus scraping targets
- ✓ kube-state-metrics deployed
- ✓ Node Exporter running on all nodes
- ✓ Grafana accessible with dashboards
- ✓ Alertmanager configured
- ✓ Alert rules defined and tested
- ✓ Notifications working (Slack/Email/PagerDuty)
- ✓ Dashboards reviewed and optimized

COMMON PROMETHEUS COMMANDS

```

# Check Prometheus status
kubectl get pods -n monitoring

# Port forward Prometheus
kubectl port-forward svc/prometheus-server -n monitoring 9090:80

# Port forward Grafana
kubectl port-forward svc/grafana -n monitoring 3000:80

# Check targets
curl http://localhost:9090/api/v1/targets
  
```

KEY TAKEAWAY

- ✓ Monitor everything important.
- ✓ Alert on what matters.
- ✓ Visualize for quick understanding.
- ✓ Use observability to find root causes fast.
- ✓ Good monitoring = Happy users.

VERIQTA

veriqta

veriqta

veriqta

@veriqta

@veriqta

@veriqta

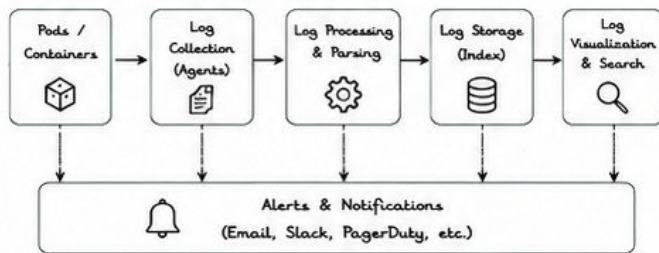
Learn. Practice. Build. Deploy. Repeat.

AT A GLANCE

- ✓ Collect logs from containers and nodes.
- ✓ Centralize, store and search logs.
- ✓ Improve troubleshooting and visibility.
- ✓ Correlate logs with metrics and traces.
- ✓ Plan retention and control costs.

“ Good logging turns problems into understanding. Centralized logs give you the full story, not just a part of it. ”

1. LOGGING ARCHITECTURE OVERVIEW



★ Logs flow: Application → Node → Agent → Pipeline → Storage → Visualization

2. LOGGING COMPONENTS

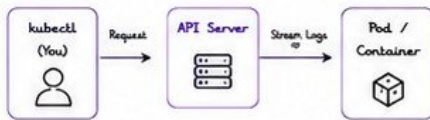
COMPONENT	ROLE
kubectl logs	On-demand logs from Pods and containers.
Log Agent (Fluent Bit / Fluentd)	Collects, buffers and forwards logs.
Log Pipeline	Filters, parses, enriches and routes logs.
Log Storage (Loki / Elasticsearch)	Stores and indexes logs for fast search.
Visualization (Grafana / Kibana)	Searches, visualizes and dashboards logs.
Alerting (Alertmanager)	Triggers alerts based on log patterns.

💡 TIP: Centralize logs early before issues become harder to trace.

3. KUBECTL LOGS

Common commands:

- `kubectl logs <pod-name>`
- `kubectl logs -f <pod-name>` # follow logs
- `kubectl logs -n <namespace> <pod-name>`
- `kubectl logs -c <container-name> <pod-name>`
- `kubectl logs --tail=100 <pod-name>`
- `kubectl logs --since=10m <pod-name>`

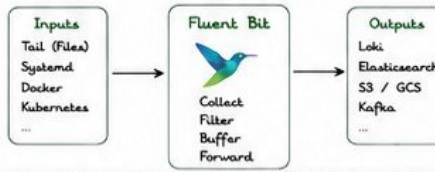


★ Use kubectl logs for quick debugging,
 not for long-term storage.

4. FLUENT BIT (LOG AGENT)

Why Fluent Bit?

- ✓ Lightweight and fast
- ✓ Low memory footprint
- ✓ Supports many inputs & outputs
- ✓ Ideal for Kubernetes and edge



Example (fluent-bit config snippet)

```

[OUTPUT]
  Name      loki
  Match     *
  Host      loki.logging.svc.cluster.local
  Port      3100
  
```

5. LOKI (LOG AGGREGATION)

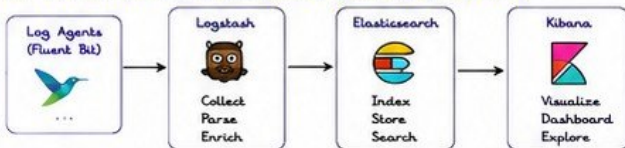
Why Loki?

- ✓ Built for logs, not full-text search
- ✓ Uses labels like Prometheus
- ✓ Cost-effective, efficient storage
- ✓ Great for Kubernetes environments



Loki labels example:
`{job="app", namespace="prod", pod="web-7d8b"}`
 Log line: "User login failed for user admin"

6. ELK STACK (ELASTICSEARCH, LOGSTASH, KIBANA)



Why ELK?

- ✓ Powerful search and analytics
- ✓ Full-text search capabilities
- ✓ Rich visualizations and dashboards
- ✓ Wide adoption and ecosystem

Typical indices:

- `app-logs-2024.05.10`
- `system-logs-2024.05.10`
- `audit-logs-2024.05.10`

7. CENTRALIZED LOGGING BEST PRACTICES

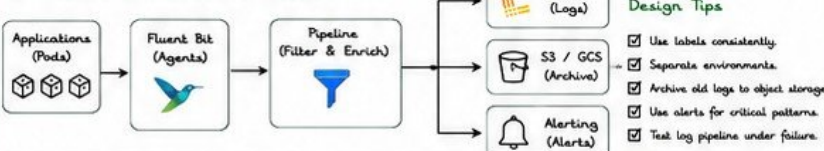
- ✓ Ship logs off the node immediately.
- ✓ Use structured JSON logs when possible.
- ✓ Include context: timestamp, level, trace_id, request_id.
- ✓ Set retention policies to control cost.
- ✓ Encrypt logs in transit and at rest.
- ✓ Monitor log pipeline health.
- ✓ Create dashboards for key services.

Structured Log Example (JSON)

```

{
  "timestamp": "2024-05-10T10:15:30Z",
  "level": "ERROR",
  "message": "Payment failed",
  "service": "payment-api",
  "trace_id": "abc123",
  "user_id": "u-789",
  "status": 500
}
  
```

8. PRODUCTION LOGGING DESIGN



Design Tips

- ✓ Use labels consistently.
- ✓ Separate environments.
- ✓ Archive old logs to object storage.
- ✓ Use alerts for critical patterns.
- ✓ Test log pipeline under failure.

9. COMMON LOG ISSUES & FIXES

ISSUE	FIX
No logs in backend	Check agent logs, network, output config.
Missing logs	Check file paths, permissions, filters.
High log volume	Add filters, reduce log level, sample logs.
Slow queries	Optimize indexes, retention, labels.
Disk full on node	Ship logs off-node, set rotation.

USEFUL COMMANDS

```

kubectl get pods -A
kubectl logs -f <pod> -c <container>
kubectl top pods
kubectl get nodes
kubectl describe pod <pod-name>
  
```

LOG PIPELINE CHECKLIST

- ✓ Log agents running on all nodes.
- ✓ Logs flowing to backend (test log).
- ✓ Dashboards and alerts configured.
- ✓ Retention and storage policies set.
- ✓ Pipeline monitored and healthy.

KEY TAKEAWAY

- ✓ Centralize logs to gain full visibility.
- ✓ Use the right tools for your scale and needs.
- ✓ Structure logs and add context.
- ✓ Monitor the pipeline, not just the apps.
- ✓ Good logging makes you a better engineer.

27. SECURITY FUNDAMENTALS

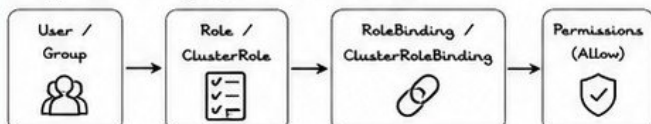
“ Security in Kubernetes is not about blocking people. It is about allowing the right people to do the right things. ”

AT A GLANCE

- ✓ Secure access to the right things.
- ✓ Use least privilege always.
- ✓ Protect the API and the data.
- ✓ Harden the cluster, not just the apps.
- ✓ Security is continuous, not one-time.

1. RBAC (ROLE BASED ACCESS CONTROL)

- Controls who can do what in the cluster.
- Based on Roles, RoleBindings, ClusterRoles, ClusterRoleBindings.
- Applies to users, groups or Service Accounts.



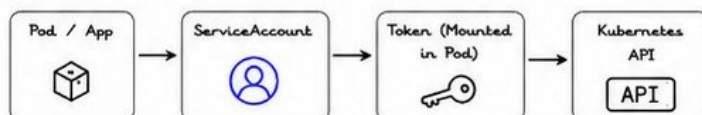
Scope:

- Role → within a namespace
- ClusterRole → across the cluster

★ TIP: Grant only the minimum permissions required.

2. SERVICE ACCOUNTS

- Used by applications and controllers to access the API.
- Each namespace gets a default Service Account.
- Bind Roles to Service Accounts (not to default unless needed).



★ Best Practice:

- Create dedicated Service Accounts per app / workload.
- Disable token automount if not needed.
- Rotate tokens and use short-lived tokens when possible.

3. LEAST PRIVILEGE PRINCIPLE

- Give users and workloads only what they need.
- Avoid cluster-admin and wild-card permissions.
- Review and audit permissions regularly.

GOOD

Role: view-pods
Rules:
- apiGroups: ["*"]
resources: ["pods"]
verbs: ["get", "list", "watch"]

Purpose: Read-only access to Pods

BAD

Role: admin
Rules:
- apiGroups: ["*"]
resources: ["*"]
verbs: ["*"]

Risk: Full access to everything!

★ TIP: Start with no access. Add what is needed.

4. API SECURITY

- The API server is the control plane gateway.
- Protect it at all layers.

Key Controls

- ✓ Use TLS everywhere (in-transit encryption)
- ✓ Authenticate using strong methods
- ✓ Authorize using RBAC
- ✓ Enable Audit Logging
- ✓ Enable API Rate Limiting
- ✓ Disable anonymous access
- ✓ Restrict access to the API server (Network / Firewall)
- ✓ Use short-lived tokens (TokenRequest API)

TIP: The API server is the most sensitive part of the cluster.

5. CLUSTER SECURITY

- Harden the cluster infrastructure.
- Reduce the attack surface.
- Continuously patch and monitor.

KEY AREAS	BEST PRACTICES
Control Plane	Isolate, restrict access, TLS, strong auth, regular backups
Worker Nodes	Harden OS, disable unnecessary services, update regularly
Network	Use NetworkPolicies, restrict pod-to-pod traffic
Secrets	Use Secrets, not ConfigMaps. Encrypt with KMS / Vault
Images	Use trusted registries, scan images, run as non-root
Etc	Enable encryption at rest, secure backups

★ TIP: Secure the platform so applications can run securely.

6. COMMON SECURITY OBJECTS

OBJECT	PURPOSE
Role / ClusterRole	Define permissions (rules)
RoleBinding / ClusterRoleBinding	Assign permissions to subjects
ServiceAccount	Identity for Pods / workloads
Secret	Store sensitive data (tokens, keys)
NetworkPolicy	Control network traffic to Pods
PodSecurity (PSA)	Enforce security standards on Pods
Admission Control	Validate and enforce policies

7. SECURITY CHECKLIST

- ✓ Use RBAC — no cluster-admin unless necessary
- ✓ Create dedicated Service Accounts
- ✓ Apply least privilege
- ✓ Protect the API server (TLS, Auth, Audit)
- ✓ Restrict API access (Network / Firewall)
- ✓ Use NetworkPolicies
- ✓ Scan container images and use non-root user
- ✓ Encrypt Secrets and etcd
- ✓ Keep nodes, components and images updated
- ✓ Monitor audit logs and events

★ TIP: Security is everyone's job, not just the platform team's job.

8. RBAC EXAMPLE

Role: read-config (within a namespace)

```

rules:
- apiGroups: ["*"]
  resources: ["configmaps", "secrets"]
  verbs: ["get", "list"]
  
```

Bind to ServiceAccount:

```

roleRef:
  kind: Role
  name: read-config
subjects:
- kind: ServiceAccount
  name: my-app-sa
  namespace: my-namespace
  
```

★ TIP: Bind to the ServiceAccount, not to the default one.

9. THINGS TO AVOID

- ✗ Using default ServiceAccount
- ✗ Giving cluster-admin access
- ✗ Using long-lived tokens
- ✗ Storing secrets in images or code
- ✗ Opening the API server to the world

10. USEFUL COMMANDS

```

# View roles
kubectl get roles -n <ns>

# View role bindings
kubectl get rolebindings -n <ns>

# View service accounts
kubectl get sa -n <ns>

# Describe a role
kubectl describe role <role-name> -n <ns>

# Auth can-i check
kubectl auth can-i get pods --as=system:serviceaccount:<ns>:sa -n <ns>
  
```

KEY TAKEAWAY

- ✓ Secure access with RBAC
- ✓ Use Service Accounts and least privilege
- ✓ Protect the API server
- ✓ Harden the cluster infrastructure
- ✓ Security is continuous and proactive



28. POD SECURITY

“ Secure Pods. Secure Platform. Secure Everything.
Pod security is your first line of defense
in Kubernetes. ”

AT A GLANCE

- ✓ Enforce Pod Security Standards.
- ✓ Run containers with least privilege.
- ✓ Control capabilities and privileges.
- ✓ Avoid root users.
- ✓ Harden Pods for production.

1. POD SECURITY OVERVIEW

- Pods are the smallest deployable units.
- Insecure Pods can escape, escalate privileges, and compromise the cluster.
- Defense starts at the Pod level.



★ TIP: Never trust containers. Always enforce security at multiple layers.

2. POD SECURITY STANDARDS (PSA)

- Kubernetes provides built-in Pod Security Standards.
- Enforced via labels on namespaces.

STANDARD	DESCRIPTION	USE CASE
Privileged	Unrestricted policy. No restrictions.	Legacy / Fully trusted workloads.
Baseline	Minimally restrictive. Prevents known privilege escalations.	General purpose workloads.
Restricted	Heavily restricted. Strong security defaults.	High security environments.

Enforcement Levels:

- enforce - Blocks violating pods
- audit - Logs warnings
- warn - Warns users

Example: Namespace Label

pod-security.kubernetes.io/enforce: restricted
pod-security.kubernetes.io/audit: baseline
pod-security.kubernetes.io/warn: baseline

3. SECURITY CONTEXT

Defines privilege and access settings for Pods and containers.

Pod Level SecurityContext Example:

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  runAsGroup: 1000
  fsGroup: 2000
  seccompProfile:
    type: RuntimeDefault
```

runAsNonRoot: Prevents containers from running as root.
runAsUser: Sets default user ID.
runAsGroup: Sets default group ID.
fsGroup: Gives group ownership of mounted volumes.
seccompProfile: Enables syscall filtering.

4. CAPABILITIES

Linux capabilities control what a container can do.

Default: Containers drop many capabilities.

Common Dangerous Capabilities (Avoid)

- ✗ NET_ADMIN
- ✗ SYS_ADMIN
- ✗ SFS_MODE
- ✗ DAC_OVERRIDE
- ✗ SETUID
- ✗ SETGID

Common Safe Capabilities

- ✓ CHOWN
- ✓ DAC_READ_SEARCH
- ✓ FOWNER
- ✓ NET_BIND_SERVICE
- ✓ SETPCAP

Example: Drop ALL, Add Only Needed

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]
```

5. AVOID ROOT USERS

Running as root increases risk of privilege escalation.

BAD: Running as root

```
securityContext:
  runAsUser: 0 # root
  runAsNonRoot: false
```



GOOD: Non-root user

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  runAsGroup: 1000
```



★ TIP: Always use minimal non-root user.

6. ADDITIONAL POD SECURITY CONTROLS

CONTROL	PURPOSE
readOnlyRootFilesystem	Prevents writing to container filesystem.
allowPrivilegeEscalation: false	Prevents gaining more privileges.
privileged: false	Disallows privileged containers.
hostNetwork: false	Prevents using host network namespace.
hostPID: false	Prevents access to host PID namespace.
hostIPC: false	Prevents access to host IPC namespace.
seccompProfile	Restricts syscalls.

★ TIP: Default deny. Only allow what is required.

7. PRODUCTION HARDENING CHECKLIST

- ✓ Use Pod Security Standards (Restricted).
- ✓ Run containers as non-root.
- ✓ Drop all capabilities and add only required ones.
- ✓ Set readOnlyRootFilesystem: true.
- ✓ Disable privilege escalation.
- ✓ Use seccompProfile: RuntimeDefault.
- ✓ Avoid host namespaces (PID, IPC, Network).
- ✓ Use minimal base images.
- ✓ Scan images for vulnerabilities.
- ✓ Keep images and dependencies updated.
- ✓ Monitor and audit Pod behavior.



8. POD SECURITY BEST PRACTICES

- Apply least privilege to every Pod.
- Use Pod Security Standards across namespaces.
- Run as non-root. Always.
- Drop dangerous capabilities.
- Continuously validate and audit.

9. EXAMPLE: SECURE POD MANIFEST (SNIPPET)

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
  labels:
    app: secure-app
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 1000
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      image: nginx:1.25-alpine
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
          add: ["NET_BIND_SERVICE"]
```

10. COMMON RISKS & IMPACT

- Running as root → Full system compromise
- Privilege escalation → Escape container
- Excess capabilities → Bypass kernel restrictions
- Writable root FS → Malware persistence
- Host namespaces → Break isolation



KEY TAKEAWAY

Secure Pods. Secure Cluster.
Start with Pod Security Standards,
run least privilege, and stay secure.

USEFUL COMMANDS

- # Check namespace PSA labels
kubectl get ns --show-labels
- # Describe Pod security context
kubectl describe pod <pod-name>

- # View Pod capabilities
kubectl exec <pod-name> --
capsh --print
- # Check PSA violations (Audit logs)
kubectl get events --field-selector type=Warning

- # Verify running user in container
kubectl exec <pod-name> -- id
- # Scan images
trivy image <image-name>



kubernetes

VERIQTA

29. IMAGE SECURITY

PAGE 29

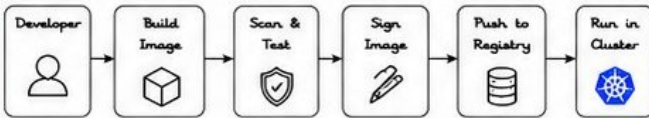
AT A GLANCE

- ✓ Use private registries.
- ✓ Store credentials with ImagePullSecrets.
- ✓ Scan images for vulnerabilities.
- ✓ Run only trusted and signed images.
- ✓ Protect the entire supply chain.

“Your container image is the blueprint of your workload.
Trust it. Verify it. Protect it.”

1. IMAGE SECURITY OVERVIEW

- Container images can contain vulnerabilities, malware or unwanted packages.
- Image security ensures only trusted, verified images run in your cluster.
- Security must be applied across the entire image lifecycle.



★ TIP: Secure the image before it ever reaches the cluster.

2. PRIVATE REGISTRIES

- Store images in private registries you control.
- Examples: Harbor, AWS ECR, Azure ACR, Google GAR, Docker Hub Private.



Why use private registries?

- ✓ Control access to images.
- ✓ Keep sensitive images out of the public internet.
- ✓ Improve reliability and reduce rate limits.
- ✓ Enable vulnerability scanning and policy enforcement.

3. IMAGEPULLSECRETS

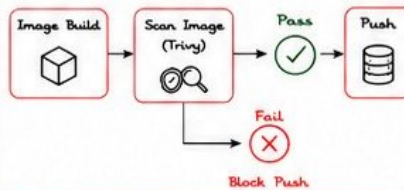
- Kubernetes uses ImagePullSecrets to authenticate to private registries.
- Secrets are stored in the namespace and attached to Pods or ServiceAccounts.



★ TIP: Never hardcode credentials in Pods or images.

4. IMAGE SCANNING

- Scan images for vulnerabilities before pushing and regularly in the registry.
- Tools: Trivy, Clair, Anchore, Gripe.

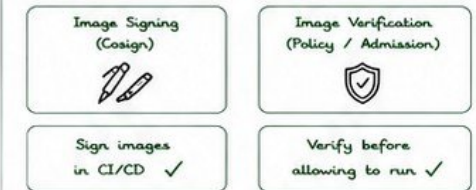


What is scanned?

- ✓ OS packages
- ✓ Application dependencies
- ✓ Known vulnerabilities (CVEs)
- ✓ Misconfigurations
- ✓ Secrets and malware

5. TRUSTED IMAGES

- Run only images you trust and approve.
- Use image signing and verification.
- Verify the publisher and source.

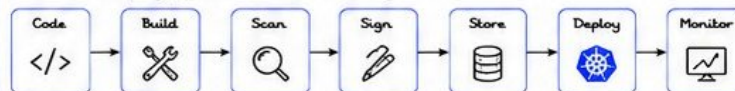


Best Practices

- ✓ Use signed images (Cosign / Notary).
- ✓ Enforce signature verification.
- ✓ Allow only approved registries.
- ✓ Maintain a trusted image list.

6. SUPPLY CHAIN SECURITY

- Protect every step from code to running container.
- Ensure integrity, provenance and traceability.



Key Controls

- ✓ Use minimal base images.
- ✓ Pin image tags by digest (not just tag).
- ✓ Scan dependencies and base images.
- ✓ Use admission policies to enforce rules.
- ✓ Continuously monitor for new vulnerabilities.

Example: Pin by Digest

image: myregistry.com/app@sha256:3f8a2...

Digest ensures the exact image is pulled.
Prevents tag hijacking and unexpected updates.

8. COMMON RISKS

- ✗ Using public registries for sensitive images.
- ✗ Storing credentials in plain text.
- ✗ Running unscanned images.
- ✗ Pulling images with :latest tag.
- ✗ Not verifying signatures.
- ✗ Allowing any registry access.
- ✗ Compromised base images.
- ✗ Broken or unknown supply chain.



9. BEST PRACTICES CHECKLIST

- ✓ Use private registries and control access.
- ✓ Store credentials in ImagePullSecrets.
- ✓ Scan images in CI/CD and registry.
- ✓ Sign images and verify before running.
- ✓ Run only trusted and minimal images.
- ✓ Pin tags by digest, not by tag.
- ✓ Enforce policies with admission controllers.
- ✓ Continuously monitor and update images.

7. PRODUCTION HARDENING

- Only run necessary images.
- Remove unused packages and tools.
- Keep OS and dependencies up to date.
- Use read-only filesystems where possible.

Hardening Checklist

- ✓ Use minimal and trusted base image.
- ✓ Run as non-root user.
- ✓ Drop unnecessary packages and shells.
- ✓ Use read-only filesystem and least privileges.
- ✓ Enable seccomp, AppArmor or SELinux.



USEFUL COMMANDS

```

# Create ImagePullSecret
kubectl create secret docker-registry regcred \
  --docker-server=myregistry.com \
  --docker-username=USER \
  --docker-password=PASS \
  --docker-email=email@domain.com

# store registry creds

# Scan image with Trivy
trivy image myregistry.com/app:1.2.3

# scan image

# Pull image by digest
kubectl run test --image=myregistry.com/app@sha256:3f8a2...

# pull exact image

# Describe pod image pull status
kubectl describe pod <pod-name>

# troubleshoot pulls
  
```

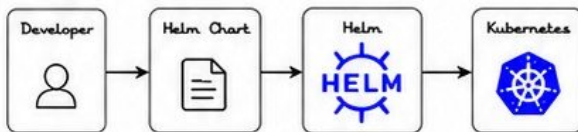
AT A GLANCE

- ✓ Package Kubernetes applications.
- ✓ Use Charts to define resources.
- ✓ Customize using Values.
- ✓ Manage Releases.
- ✓ Upgrade and rollback safely.

Helm is the package manager for Kubernetes.
Package, configure, install, upgrade,
and manage Kubernetes applications.

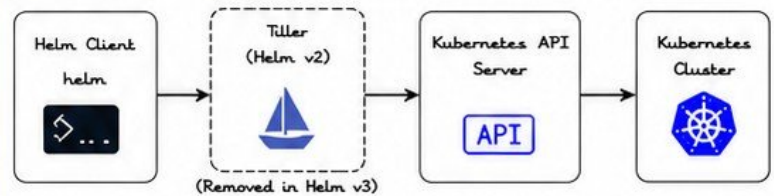
1. HELM OVERVIEW

- Helm helps you define, install, and upgrade even the most complex Kubernetes applications.
- It uses Charts to package Kubernetes manifests.
- Releases are instances of charts running in your cluster.



TIP: Helm does not replace Kubernetes. It makes working with Kubernetes easier.

2. HELM ARCHITECTURE



- Helm v2 used Tiller (server-side component).
- Helm v3 is client-only and talks directly to the API server.
- Helm stores release information in the cluster (Secrets).

3. CHARTS

- A Chart is a package of pre-configured Kubernetes resources.
- Charts have a specific directory structure.

```

mychart/
├── Chart.yaml      # Metadata about the chart
├── values.yaml     # Default configuration values
├── charts/         # Chart dependencies
├── templates/      # Kubernetes manifest templates
│   ├── deployment.yaml
│   ├── service.yaml
│   └── _helpers.tpl
└── templates/NOTES.txt # Info shown after install
  
```

TIP: Keep charts modular and reusable. Avoid hardcoding values.

4. VALUES

- values.yaml holds configurable parameters.
- You can override values at install/upgrade.

Example values.yaml

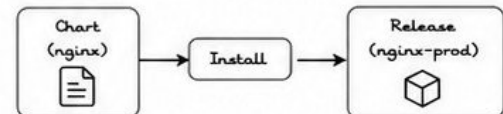
```

replicaCount: 2
image:
  repository: nginx
  tag: "1.25"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
  requests:
    cpu: "100m"
    memory: "128Mi"
  
```

TIP: Use values files per environment (dev, staging, prod).

5. RELEASES

- A Release is a running instance of a chart.
- Each release has a unique name and version.
- Helm tracks releases and their history.



Release History Example

REVISION	UPDATED	STATUS	DESCRIPTION
3	2024-07-01 10:30	deployed	Upgrade complete
2	2024-07-01 09:15	superseded	Upgrade complete
1	2024-07-01 08:00	superseded	Install complete

TIP: Use meaningful release names. It helps in managing and debugging.

6. INSTALLING A CHART

```
helm install myapp ./mychart -f values-prod.yaml
```

release name chart path custom values

Common Flags

- `--namespace` Install in specific namespace
- `--create-namespace` Create namespace if not exists
- `--set key=value` Override a single value
- `--wait` Wait for all resources to be ready
- `--atomic` Rollback on failure

TIP: Always test charts in non-production first.

7. UPGRADES

```
helm upgrade myapp ./mychart -f values-prod.yaml
```

release name chart updated values

Best Practices

- Always review changes before upgrading.
- Use `--dry-run --debug` to preview changes.
- Use version control for your values files.
- Test upgrades in staging before production.

TIP: Helm keeps history. You can rollback if needed.

8. ROLLBACKS

```
helm rollback myapp 2
```

release revision number

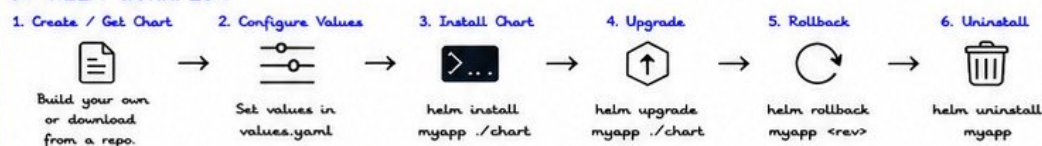
Other Useful Commands

```

helm list           # List releases
helm status myapp  # Show release status
helm history myapp # Show release history
helm uninstall myapp # Uninstall a release
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update   # Update repo index
helm search repo nginx # Search charts
  
```

TIP: Clean up old releases and unused resources.

9. HELM WORKFLOW



Key Takeaway

Helm brings structure, reusability, and control to Kubernetes deployments. Master your charts. Ship with confidence.

31. GITOPS INTRODUCTION

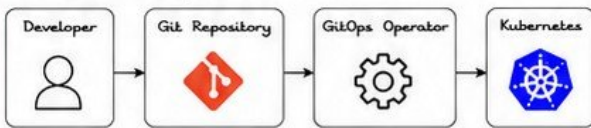
“GitOps changes how we operate Kubernetes.
Git is the source of truth.
The cluster reflects the desired state.”

AT A GLANCE

- ✓ Git is the source of truth.
- ✓ Declarative desired state.
- ✓ Continuous reconciliation.
- ✓ Automated, auditable, reliable.
- ✓ Safer deployments at scale.

1. GITOPS INTRODUCTION

- GitOps is a way to manage infrastructure and applications using Git as the single source of truth.
- All changes are stored, reviewed, and versioned in Git.
- Tools continuously reconcile the cluster to match the desired state in Git.
- It brings automation, safety, and auditability to Kubernetes operations.



★ TIP: Git is not just for code anymore.
It is for infrastructure, too.

2. GITOPS PRINCIPLES

	Declarative	Describe the desired state in files. The system makes it happen.
	Git as Source of Truth	All environment configurations live in Git. Git is immutable, versioned, and auditable.
	Automation	Automate the deployment and reconciliation of changes.
	Continuous Reconciliation	The system continuously compares the cluster state with Git and fixes drift.
	Auditability	Every change is tracked, reviewable, and traceable.

3. DESIRED STATE

- The desired state is defined in Git as declarative files (YAML, Kustomize, Helm, etc).
- This state represents how the cluster should look.
- The GitOps operator ensures the cluster always matches this state.

Examples of desired state files

- Deployment YAML
- Service YAML
- Ingress YAML
- ConfigMaps, Secrets
- Helm values
- Kustomize overlays

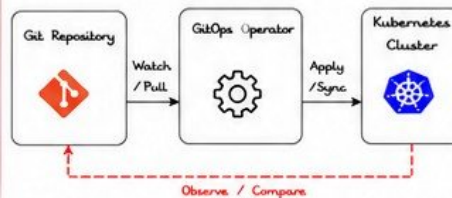
```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  ...
  
```

★ TIP: If it's not in Git, it doesn't exist.

4. CONTINUOUS RECONCILIATION

- The GitOps operator runs in the cluster.
- It watches the Git repository for changes.
- Whenever there is a change, it compares the desired state (Git) with the live state (cluster).
- If there is a difference, it automatically reconciles the cluster to match Git.



★ TIP: Drift happens.
Reconciliation fixes it automatically.

5. GIT WORKFLOWS

- Teams use Git workflows for safe and controlled changes.
- Common GitOps workflow:



6. BENEFITS OF GITOPS

	Version Control All changes are versioned and traceable.
	Auditability Who changed what, when, and why is always known.
	Consistency Environments stay consistent and repeatable.
	Automation Reduces manual work and human error.
	Recovery Easy rollback by reverting Git changes.
	Security RBAC, reviews, and policies enforce safe changes.
	Scalability Works across clusters, regions, and clouds.

7. GITOPS ECOSYSTEM TOOLS

Popular GitOps tools:

-  **Argo CD**
Declarative GitOps CD for Kubernetes.
-  **Flux CD**
GitOps toolkit for Kubernetes.
-  **Rancher Fleet**
GitOps at scale for multi-cluster.
-  **Helm + Git**
Helm charts versioned in Git.
-  **Kustomize + Git**
Kustomize configurations in Git.

8. KEY TAKEAWAY

- ✓ GitOps uses Git as the source of truth.
- ✓ You define the desired state declaratively.
- ✓ The system continuously reconciles the real state to match Git.
- ✓ Safe, automated, auditable, and reliable.
- ✓ Small change in Git → Big impact in the cluster.

Remember

Git stores the truth.
GitOps keeps the cluster honest.
You focus on changes,
GitOps handles the operations.



kubernetes

32. TROUBLESHOOTING KUBERNETES

“

Every outage has signals.

Use the right commands.

Follow the clues. Fix the root cause.

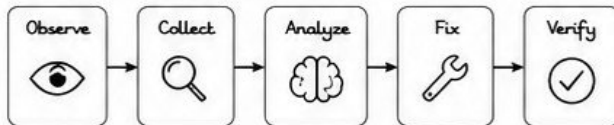
”

AT A GLANCE

- ✓ Observe the system.
- ✓ Collect the right data.
- ✓ Find the root cause.
- ✓ Fix it and verify.
- ✓ Document and prevent.

1. TROUBLESHOOTING MINDSET

- Start broad. Narrow down.
- Observe → Understand → Act → Verify
- Never guess. Always validate.
- The system provides signals. Learn to read them.
- Check recent changes first.



★ TIP: Kubernetes rarely lies.
It gives you the truth in events, logs, and status.

2. kubectl describe

- Best first command for any Kubernetes object.
- Shows status, events, relationships, and errors.

```
# Describe a pod
kubectl describe pod <pod-name> -n <namespace>

# Describe a deployment
kubectl describe deploy <deploy-name> -n <namespace>

# Describe a node
kubectl describe node <node-name>
```

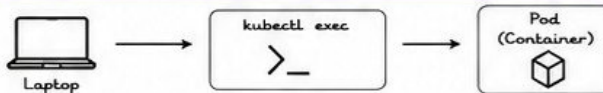
★ TIP: Events are at the bottom.
They tell you what Kubernetes is trying to tell you.

5. EXEC INTO POD

- Use exec to inspect the pod from inside.
- Great for debugging network, files, and processes.

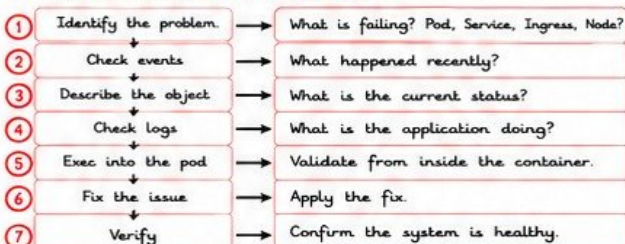
```
# Open a shell
kubectl exec -it <pod-name> -n <namespace> -- /bin/bash

# Run a single command
kubectl exec <pod-name> -n <namespace> -- <command>
```



★ TIP: Minimal images may not have bash.
Try /bin/sh if bash is not available.

7. TROUBLESHOOTING WORKFLOW



3. EVENTS

- Events show what happened in the cluster.
- They are your timeline.
- Look for Warning and Error events.

```
# Get events in a namespace
kubectl get events -n <namespace> --sort-by=.lastTimestamp

# Get events for a specific pod
kubectl get events --field-selector involvedObject.name=<pod-name> \
-n <namespace> --sort-by=.lastTimestamp
```

Common Event Types

TYPE	EXAMPLE
Normal	Scheduled, Pulled, Created, Started
Warning	FailedScheduling, FailedMount, BackOff
Error	ImagePullBackOff, CrashLoopBackOff, OOMKilled

4. LOGS

- Logs show what your application is doing.
- Always check the right container and time.

```
# Logs from a pod
kubectl logs <pod-name> -n <namespace>

# Logs from a specific container
kubectl logs <pod-name> -c <container-name> -n <namespace>

# Follow logs
kubectl logs -f <pod-name> -n <namespace>
```

★ TIP: Check logs before and after the issue started.
Correlate with events and metrics.

6. COMMON FAILURES & HOW TO INVESTIGATE

ISSUE	POSSIBLE CAUSE	WHERE TO CHECK	HOW TO START
Pod Pending	Insufficient resources Node selector mismatch PVC not bound	Events describe pod	kubectl describe pod kubectl get events
CrashLoopBackOff	App crash Bad config Dependency down	Logs describe pod	kubectl logs -f kubectl describe pod
ImagePullBackOff	Image not found Auth issue Bad image name	Events describe pod	kubectl describe pod kubectl get events
OOMKilled	Memory limits too low Memory leak	Logs describe pod	kubectl logs -f kubectl describe pod
FailedMount	PVC not available Permission issue	Events describe pod	kubectl describe pod kubectl get events

8. USEFUL CHEAT SHEET

```
kubectl get pods -A
kubectl get nodes
kubectl get all -A
kubectl get events -A --sort-by=.lastTimestamp
kubectl describe pod <pod> -n <ns>
kubectl logs -f <pod> -n <ns>
kubectl exec -it <pod> -n <ns> -- /bin/bash
kubectl top pods -n <ns>
kubectl top nodes
kubectl get pvc -A
```

★ TIP: Document the root cause and the fix.
Every incident is a learning opportunity.

Troubleshooting
is a skill.
The more you
practice,
the faster
you solve.



kubernetes

33. PRODUCTION FAILURES

“

Failures are normal in production.
How you detect, understand, and recover
defines a reliable platform.

”

AT A GLANCE

- ✓ Know the common failures.
- ✓ Understand the root cause.
- ✓ Use the right commands.
- ✓ Recover and prevent reoccurrence.
- ✓ Build resilient systems.

1. PENDING PODS

WHAT IT MEANS

The pod is accepted by the cluster but cannot be scheduled on a node.

COMMON CAUSES

- Insufficient resources (CPU/Memory)
- Node selector / affinity mismatch
- Taints not tolerated
- PVC not bound
- Too many pods (IP exhaustion)

HOW TO CHECK

- `kubectl get pods -A`
- `kubectl describe pod <pod>`
- `kubectl get nodes`
- `kubectl describe node <node>`
- `kubectl get events --sort-by=.lastTimestamp`

KEY INDICATORS

STATUS = Pending
Events show scheduling failure.
No node assigned.

HOW TO FIX

- Check node resources.
- Verify selectors, affinity, taints.
- Ensure PVC is bound.
- Add tolerations if needed.
- Scale nodes or free resources.



Events will always tell you why.

2. CRASHLOOPBACKOFF

WHAT IT MEANS

The container starts, crashes, and Kubernetes keeps restarting it.

COMMON CAUSES

- Application error
- Bad configuration
- Missing dependencies
- Probe misconfiguration
- Out of memory

HOW TO CHECK

- `kubectl get pods`
- `kubectl describe pod <pod>`
- `kubectl logs <pod> --previous`
- `kubectl logs <pod>`
- `kubectl get events --sort-by=.lastTimestamp`

KEY INDICATORS

STATUS = CrashLoopBackOff
RESTARTS keep increasing.
Events show container crash reason.

HOW TO FIX

- Check logs for the real error.
- Fix configuration or code.
- Verify probes and resources.
- Ensure dependencies are available.
- Redeploy and monitor.



Logs are your first place to look.

3. IMAGEPULLBACKOFF

WHAT IT MEANS

Kubernetes cannot pull the container image from the registry.

COMMON CAUSES

- Image name is wrong
- Image does not exist
- Private registry authentication
- Network or DNS issues
- Registry unavailable

HOW TO CHECK

- `kubectl describe pod <pod>`
- `kubectl get pods`
- `kubectl get events --sort-by=.lastTimestamp`

KEY INDICATORS

STATUS = ImagePullBackOff
Events show image pull errors.
Back-off increasing.

HOW TO FIX

- Verify image name and tag.
- Check registry access.
- Create or update ImagePullSecret.
- Check network / DNS.
- Ensure registry is available.



ImagePullSecrets solve private registry issues.

4. OOMKILLED

WHAT IT MEANS

The container is killed because it used more memory than allowed.

COMMON CAUSES

- Memory limit too low
- Memory leak in application
- High memory usage
- Incorrect resource requests

HOW TO CHECK

- `kubectl describe pod <pod>`
- `kubectl logs <pod> --previous`
- `kubectl top pod <pod>`
- `kubectl top node <node>`

KEY INDICATORS

STATUS = OOMKilled
Exit code 137
Reason = OOMKilled

HOW TO FIX

- Increase memory limit.
- Optimize application memory usage.
- Add resource requests and limits.
- Monitor with metrics.
- Tune JVM / runtime settings.



Monitor memory usage before increasing limits.

5. NODE NOTREADY

WHAT IT MEANS

The node is not ready and cannot accept pods.

COMMON CAUSES

- Node crash or shutdown
- Network issues
- Disk pressure
- Kubelet / system failure
- Cloud provider issue

HOW TO CHECK

- `kubectl get nodes`
- `kubectl describe node <node>`
- `kubectl get events --sort-by=.lastTimestamp`
- `kubectl get pods -o wide`

KEY INDICATORS

STATUS = NotReady
KUBELET not reporting.
Events show node issues.

HOW TO FIX

- Check node health.
- Restart kubelet if needed.
- Fix disk / memory / network.
- Replace or reboot node.
- Check cloud provider status.



No ready node, no running pods.

BEST PRACTICES TO PREVENT FAILURES

- ✓ Set proper resource requests and limits.
- ✓ Use liveness and readiness probes.
- ✓ Monitor everything: nodes, pods, apps.
- ✓ Keep images updated and scanned.
- ✓ Use GitOps and automated rollbacks.
- ✓ Test failure scenarios in staging.



Remember:
Expect failures.
Design for recovery.
Build for resilience.

34. HIGH AVAILABILITY

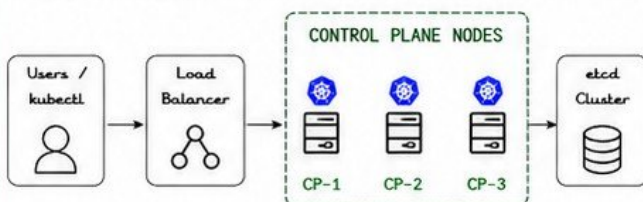
“High availability is not a feature.
It is a design.
Build for failure. Operate with confidence.”

AT A GLANCE

- ✓ Design control plane for HA.
- ✓ Run multi-node clusters.
- ✓ Backup etcd regularly.
- ✓ Prepare for failure recovery.
- ✓ Follow HA best practices.

1. HIGHLY AVAILABLE CONTROL PLANE

- Single control plane is a single point of failure.
- Run multiple control plane nodes across different availability zones.
- Use a load balancer to distribute requests.
- Keep etcd highly available.

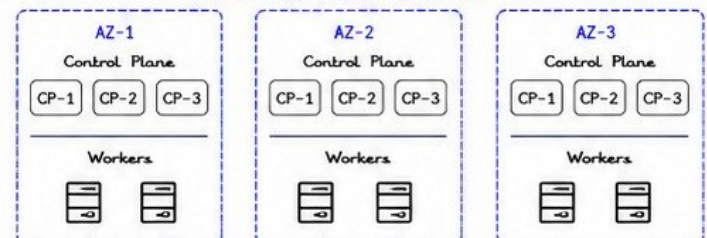


★ TIP: Always run at least 3 control plane nodes for production clusters.

2. MULTI-NODE CLUSTERS

- Run worker nodes across multiple availability zones.
- Avoid single points of failure.
- Ensure even pod distribution for resilience.
- Use PodDisruptionBudget to protect critical workloads.

MULTI-NODE, MULTI-AZ CLUSTER



★ TIP: Spread control plane and worker nodes across at least 3 availability zones.

3. ETCD BACKUP

- etcd stores the entire cluster state.
- Regular backups are critical.
- Test restore process periodically.

BACKUP COMMAND

```
etcdctl snapshot save /backup/etcd-snapshot.db
```

BACKUP STRATEGY

ITEM	RECOMMENDATION
Frequency	Daily or more frequent
Storage	Store off-cluster (S3, NFS, etc.)
Retention	Keep multiple versions
Test	Regularly test restore process

★ TIP: Without etcd backup, recovery is difficult and data loss is possible.

4. FAILURE RECOVERY

- Be prepared for node, AZ, or control plane failures.
- Have a clear recovery plan.
- Practice recovery in a staging environment.

COMMON FAILURES AND RECOVERY

FAILURE	RECOVERY ACTION
Control Plane Node Down	Other CP nodes take over automatically.
Worker Node Down	Pods reschedule on healthy nodes.
AZ Failure	Workloads continue in remaining AZs.
etcd Failure	Restore from latest snapshot to new node.
Cluster Disaster	Rebuild cluster using backups and IaC.

★ TIP: Document recovery steps and keep backups of manifests and configurations.

5. HIGH AVAILABILITY BEST PRACTICES

- ✓ Use at least 3 control plane nodes.
- ✓ Distribute nodes across multiple availability zones.
- ✓ Use a reliable load balancer for API server.
- ✓ Enable etcd backup and test restore regularly.
- ✓ Monitor control plane and etcd health.
- ✓ Use PodDisruptionBudget for critical workloads.
- ✓ Keep worker nodes stateless and resilient.
- ✓ Automate cluster provisioning with IaC.
- ✓ Test failure scenarios in staging.
- ✓ Keep documentation up to date.

HA CHECKLIST

- ✓ 3+ control plane nodes
- ✓ Multi-AZ deployment
- ✓ Load balancer for API server
- ✓ etcd backup and restore tested
- ✓ Monitoring and alerting enabled
- ✓ Failure recovery plan documented
- ✓ Regular disaster recovery drills

High availability is not about avoiding failure. It is about continuing service when failure inevitably happens.

35. PRODUCTION ARCHITECTURE

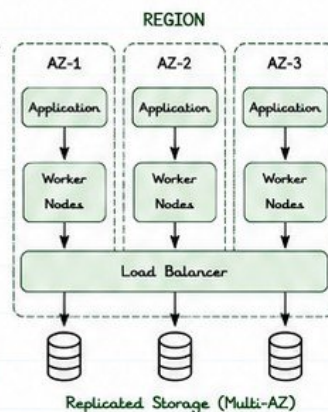
“ Production architecture is intentional.
It is designed for failure.
It is built to keep serving. ”

AT A GLANCE

- ✓ Design for reliability.
- ✓ Distribute across AZs.
- ✓ Secure by default.
- ✓ Observe everything.
- ✓ Automate and standardize.

1. MULTI-AZ DESIGN

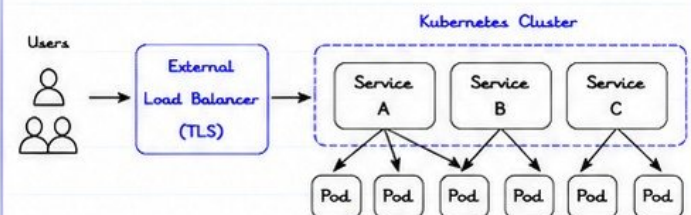
- Spread control plane and workloads across multiple Availability Zones.
- Prevent single AZ failure from affecting the service.
- Ensure high availability and resilience.



TIP: Never run all components in a single AZ. Always distribute.

2. LOAD BALANCING

- Use load balancers to distribute traffic.
- Terminate TLS at the edge.
- Perform health checks and route only to healthy pods.
- Support horizontal scaling.



TIP: Use multiple load balancers across AZs for resilience.

3. STORAGE ARCHITECTURE

- Use persistent storage for stateful workloads.
- Replicate data across AZs.
- Take regular backups and test restores.
- Use appropriate storage classes.

VOLUMES



STRATEGY

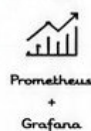
- Use replicated volumes.
- Enable snapshots.
- Backup to object storage.
- Encrypt at rest.
- Test restore regularly.

TIP: Backups are useless until you have tested recovery.

4. MONITORING

- Monitor cluster, applications, and infrastructure.
- Collect metrics, logs, and alerts.
- Create dashboards and set alert thresholds.
- Detect issues before users are impacted.

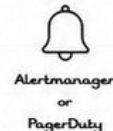
METRICS



LOGS



ALERTS



DASHBOARDS



TIP: If you don't measure it, you can't improve it.
If you don't alert on it, you will miss it.

5. SECURITY ARCHITECTURE

- Secure the cluster, workloads, and data.
- Follow the principle of least privilege.
- Encrypt in transit and at rest.
- Continuously scan and audit.

NETWORK SECURITY



- Network Policies
- VPC / Subnets
- Security Groups
- Private Endpoints
- WAF / Ingress TLS

ACCESS CONTROL



- RBAC for Users
- Service Accounts
- Least Privilege
- OIDC / SSO
- Audit Logging

SECRETS MANAGEMENT



- Kubernetes Secrets
- External Secrets
- HashiCorp Vault
- Secrets Rotation
- Encrypt Secrets

IMAGE SECURITY



- Image Scanning
- Signed Images
- Private Registry
- Vulnerability Scan
- Admission Control

SECURITY CHECKLIST

- ✓ TLS everywhere
- ✓ RBAC enabled
- ✓ Network policies enforced
- ✓ Images scanned
- ✓ Secrets encrypted
- ✓ Audit logging on

TIP: Security is not a product. It is an architecture decision.
Build it in from the start.

6. PRODUCTION ARCHITECTURE PRINCIPLES



36. KUBERNETES ON AWS (EKS)

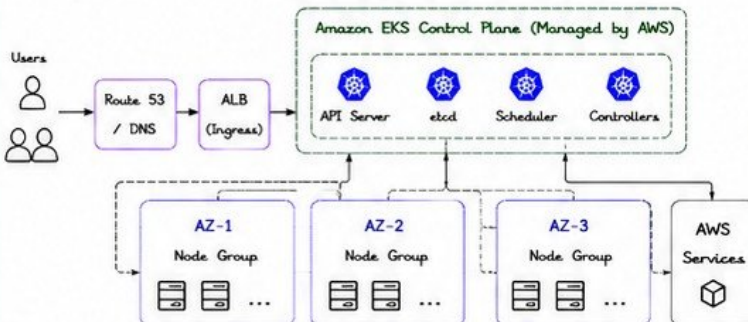
“ EKS is a managed Kubernetes service that runs Kubernetes control plane on AWS. You manage your nodes. AWS manages the control plane. ”

AT A GLANCE

- ✓ Managed control plane by AWS.
- ✓ You manage worker nodes.
- ✓ Deep AWS service integration.
- ✓ Secure with IAM and RBAC.
- ✓ Highly available by design.

1. EKS ARCHITECTURE

- AWS manages the control plane (etcd, API server, scheduler, controller).
- Runs across multiple AZs for high availability.
- You create and manage worker nodes.
- Integrates with IAM, VPC, CloudWatch, ALB, EBS, EFS, S3, etc.



TIP: The control plane is managed by AWS and is not accessible by you. Focus on securing and managing your worker nodes.

2. NODE GROUPS

- Worker nodes are organized into node groups.
- Each node group can have different instance types, sizes, and scaling.
- Supports managed node groups and self-managed node groups.
- Use Auto Scaling Group (ASG) for scaling.

NODE GROUP EXAMPLE

Node Group: general

- Instance Type: t3.large
- Desired: 3
- Min: 1
- Max: 10
- Disk: gp3
- Labels: workload=general

Node Group: compute

- Instance Type: m5i.xlarge
- Desired: 2
- Min: 1
- Max: 8
- Disk: gp3
- Labels: workload=compute

TIP: Use multiple node groups for different workloads and scaling needs.

3. IAM ROLES

- EKS uses IAM for authentication and authorization.
- Cluster Role: Allows EKS to manage AWS resources.
- Node Role: Allows worker nodes to access AWS services.
- Use IAM Roles for Service Accounts (IRSA) for pods.

CLUSTER ROLE (CREATED BY EKS)

- Allows EKS to create and manage AWS resources on your behalf.
- Do not delete or modify.

NODE ROLE (ATTACHED TO EC2)

- Attached to worker nodes.
- Allows nodes to pull from ECR, write logs to CloudWatch, access EBS, S3, etc.
- Follow least privilege.

TIP: Use IAM Roles for Service Accounts (IRSA) to give fine-grained permissions to your applications.

4. LOAD BALANCER CONTROLLER

- AWS Load Balancer Controller provisions ALB/NLB for Ingress.
- Watches Ingress resources and creates AWS Load Balancers.
- Supports SSL/TLS, path-based routing, and annotations.
- Requires IAM permissions via IRSA.

INGRESS FLOW WITH AWS LOAD BALANCER CONTROLLER



TIP: Install AWS Load Balancer Controller using Helm. Ensure IRSA is configured for least privilege.

5. STORAGE INTEGRATION

- EKS supports multiple storage options.
- Use EBS for block storage.
- Use EFS for shared file storage.
- Use S3 for object storage via CSI driver.
- Always use StorageClass for dynamic provisioning.

STORAGE TYPE	USE CASE	PROVISIONER / DRIVER
EBS (gp3, io1, st1)	Block storage for StatefulSets	ebs.csi.aws.com
EFS	Shared file storage (RWX)	efs.csi.aws.com
S3 (via CSI)	Object storage for apps	s3.csi.aws.com
FSx for Lustre	High performance workloads	fsx.csi.aws.com

TIP: Always define StorageClass and use dynamic provisioning. Avoid static provisioning unless required.

6. BEST PRACTICES FOR EKS ON AWS

- ✓ Use multiple AZs for control plane and worker nodes.
- ✓ Use managed node groups for easier scaling and upgrades.
- ✓ Enable Cluster Autoscaler for automatic scaling.
- ✓ Use IAM Roles for Service Accounts (IRSA).
- ✓ Enable VPC Flow Logs and CloudWatch Logs.
- ✓ Use AWS Load Balancer Controller for Ingress.
- ✓ Backup etcd is managed by AWS. Focus on app data backups.
- ✓ Keep worker nodes and add-ons updated.
- ✓ Use Security Groups for network isolation.
- ✓ Follow least privilege for all IAM roles.

TIP: High availability is a design. Monitor, automate, and test your platform regularly.

37. PRODUCTION CI/CD

“Great applications are built by teams.
Great platforms are built by pipelines.
Automate, test, deploy, repeat.”

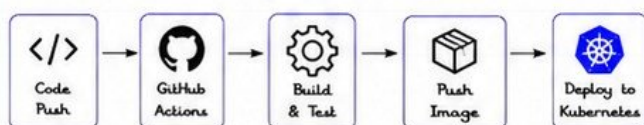
AT A GLANCE

- ✓ Automate your delivery.
- ✓ Test early. Deploy confidently.
- ✓ Use GitOps for consistency.
- ✓ Deploy safely with rollbacks.
- ✓ Pipelines are code. Version them.

1. GITHUB ACTIONS

- CI/CD platform built into GitHub.
- Great for cloud-native and microservices.
- Use workflows defined in YAML.
- Strong ecosystem and marketplace.

WORKFLOW EXAMPLE



KEY FEATURES

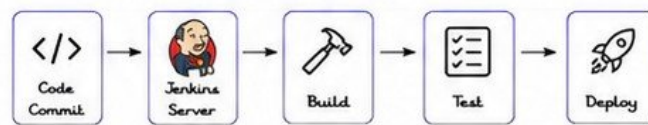
- Hosted runners
- Matrix builds
- Secrets management
- Environments
- Manual approvals
- Reusable workflows

TIP: Use environments and protection rules for safe deployments.

2. JENKINS

- Open source automation server.
- Highly customizable with plugins.
- Ideal for enterprise and complex workflows.
- Can run on-prem or in the cloud.

JENKINS PIPELINE FLOW



KEY FEATURES

- Declarative & Scripted pipelines
- Parallel builds
- Plugin ecosystem
- Credentials store
- Distributed builds
- Notifications

TIP: Keep your Jenkinsfile in Git. Pipeline as Code.

3. ARGO CD (GITOPS CD)

- Declarative, GitOps continuous delivery tool.
- Syncs Kubernetes manifests from Git to clusters.
- Detects drift and auto-corrects.
- Secure, auditable, and reliable.

ARGO CD ARCHITECTURE



KEY FEATURES

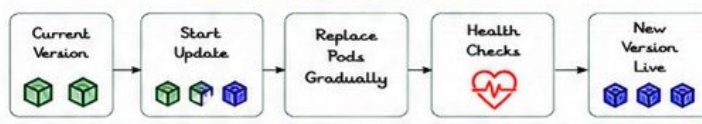
- GitOps workflow
- Auto sync + self heal
- Multi-cluster support
- Web UI
- RBAC integration
- Sync waves & hooks

TIP: One source of truth (Git). No manual kubectl apply.

4. ROLLING DEPLOYMENTS

- Update applications with zero downtime.
- Kubernetes handles rolling update strategy.
- Update pods gradually.
- Health checks ensure stability.

ROLLING DEPLOYMENT FLOW



STRATEGY SETTINGS (EXAMPLE)

- maxUnavailable: 1 (how many pods can be down)
- maxSurge: 1 (how many extra pods can be created)
- readinessProbe: /health (check before traffic)
- livenessProbe: /live (kill unhealthy pods)

TIP: Always set resource requests/limits and probes.

5. ROLLBACK AUTOMATION

- Rollback automatically on failure.
- Use health checks, metrics, and alerts.
- Argo CD and Kubernetes support automated rollbacks.
- Reduce MTTR and customer impact.

ROLLBACK OPTIONS

KUBERNETES

- kubectl rollout undo deployment/<name>
- Rollback to previous ReplicaSet
- Revision history
- Rollback with helm rollback

ARGO CD

- Auto rollback on health check failure
- Sync windows & hooks
- One-click rollback from UI
- Rollback to specific Git commit

TIP: Test rollback as much as you test deployments.

6. PRODUCTION CI/CD BEST PRACTICES

- ✓ Keep pipeline code in Git.
- ✓ Automate build, test, and security scans.
- ✓ Use artifact repository for builds.
- ✓ Use environments (dev, staging, prod).
- ✓ Require manual approval for prod.
- ✓ Use progressive delivery.
- ✓ Monitor deployments.
- ✓ Alert on failures.
- ✓ Implement auto rollback.
- ✓ Keep secrets in a secure store.
- ✓ Document pipeline steps.
- ✓ Continuously improve pipeline speed.

TIP: A good pipeline is fast, safe, repeatable, and observable.

38. PRODUCTION BEST PRACTICES

“ Best practices are not about following rules. They are about building systems that are secure, reliable, efficient, and easy to operate. ”

AT A GLANCE

- ✓ Optimize resources and costs.
- ✓ Secure everything by default.
- ✓ Design for reliability and resilience.
- ✓ Automate operations and recovery.
- ✓ Measure, improve, and iterate.

1. RESOURCE OPTIMIZATION

- Set requests and limits for all containers.
- Right-size workloads based on actual usage.
- Use Horizontal Pod Autoscaler (HPA).
- Use Vertical Pod Autoscaler (VPA) where possible.
- Clean up unused resources regularly.

BEST PRACTICES

- ✓ Set CPU and memory requests.
- ✓ Set reasonable limits.
- ✓ Use cluster autoscaler.
- ✓ Monitor and adjust continuously.
- ✓ Remove unused namespaces, PVs, and images.

KEY TOOLS

-  HPA / VPA
-  Cluster Autoscaler
-  Prometheus
-  Kubecttl / Metrics Server

★ TIP: Right-sizing saves money and prevents performance issues.

2. SECURITY HARDENING

- Use least privilege (RBAC).
- Run as non-root users.
- Scan images for vulnerabilities.
- Keep Kubernetes and dependencies updated.
- Encrypt secrets and use sealed secrets.

BEST PRACTICES

- ✓ Enforce RBAC policies.
- ✓ Use Network Policies.
- ✓ Enable Pod Security Standards.
- ✓ Scan images in CI/CD pipeline.
- ✓ Rotate secrets regularly.

KEY TOOLS

-  RBAC / PSP / PSS
-  Network Policies
-  Trivy / Gype
-  Sealed Secrets / External Secrets

★ TIP: Assume breach. Secure every layer.

3. COST OPTIMIZATION

- Use Spot Instances for non-critical workloads.
- Use right instance types and sizes.
- Schedule non-critical workloads off-peak.
- Use Kubernetes Cluster Autoscaler.
- Monitor and reduce storage and egress costs.

BEST PRACTICES

- ✓ Use Spot Instances.
- ✓ Stop idle environments.
- ✓ Use minimal storage classes.
- ✓ Optimize log retention.
- ✓ Monitor AWS cost reports.

KEY TOOLS

-  AWS Cost Explorer
-  Karpenter / Cluster Autoscaler
-  S3 Lifecycle Policies
-  CloudWatch / Prometheus

★ TIP: Optimize continuously. Unused resources cost real money.

4. RELIABILITY

- Use multiple AZs for high availability.
- Use liveness and readiness probes.
- Set PodDisruptionBudgets (PDB).
- Use replication and self-healing.
- Backup etcd and critical data.

BEST PRACTICES

- ✓ Run control plane across AZs.
- ✓ Use anti-affinity rules.
- ✓ Enable PDB for critical apps.
- ✓ Use Multi-AZ databases.
- ✓ Test failure scenarios.

KEY TOOLS

-  Liveness / Readiness Probes
-  PodDisruptionBudget
-  RDS / Aurora Multi-AZ
-  Velero / Stash (Backups)

★ TIP: Design for failure. Test it. Improve it.

5. OPERATIONAL EXCELLENCE

- Automate everything you can.
- Use GitOps and Infrastructure as Code.
- Centralize logging and monitoring.
- Document runbooks and procedures.
- Review and improve regularly.

BEST PRACTICES

- ✓ Use GitOps for deployments.
- ✓ Automate backups and restores.
- ✓ Centralize logs and metrics.
- ✓ Create runbooks for incidents.
- ✓ Run regular game days.

KEY TOOLS

-  Argo CD / Flux
-  Terraform / CloudFormation
-  Loki / Prometheus / Grafana
-  Runbooks / Wiki

★ TIP: Good operations are built, not accidental.

39. SENIOR KUBERNETES INTERVIEW MASTERCLASS

“ Senior engineers don't just know Kubernetes. They understand how to design, troubleshoot, and run it in production at scale. Think like an SRE. Solve like an architect. ”

AT A GLANCE

- ✓ Real production questions.
- ✓ System design discussions.
- ✓ Troubleshooting scenarios.
- ✓ Expected answers.
- ✓ Interview tips that work.

1. REAL PRODUCTION QUESTIONS

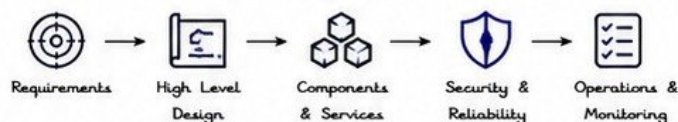
- How would you troubleshoot a high CPU usage in a Node?
- A deployment is not rolling out. How do you investigate?
- How do you handle a Pod stuck in Terminating state?
- How would you upgrade a Kubernetes cluster with zero downtime?
- How do you secure a Kubernetes cluster in production?
- How do you design for multi-tenancy in Kubernetes?
- How do you handle secrets in Kubernetes?
- How do you ensure high availability for etcd?

★ TIP: Answer with clarity, structure, and real experience. Use examples from production.

2. SYSTEM DESIGN DISCUSSIONS

- Design a highly available Kubernetes cluster on AWS.
- Design a CI/CD platform using Kubernetes.
- Design a logging and monitoring solution for Kubernetes.
- Design a multi-tenant Kubernetes platform.
- Design a backup and disaster recovery strategy.
- Design a GitOps based deployment workflow.

DESIGN FRAMEWORK



★ TIP: Clarify requirements. Make trade-offs. Communicate your design decisions.

3. TROUBLESHOOTING SCENARIOS

- Pod is running but application is not reachable.
- CrashLoopBackOff in production.
- Node NotReady and workloads are down.
- PVC is stuck in Pending state.
- API server is slow or timing out.
- DNS resolution is failing in the cluster.
- High memory usage across worker nodes.
- etcd is unhealthy.
- Ingress is not routing traffic.
- ImagePullBackOff in private registry.

★ TIP: Follow a systematic approach. Check events, metrics, logs, and configurations.

4. EXPECTED ANSWERS (WHAT INTERVIEWERS LOOK FOR)

COMPETENCY	WHAT A STRONG ANSWER INCLUDES
 Problem Solving	Clear problem breakdown, root cause analysis, and resolution steps.
 Architecture Thinking	Scalable, reliable, secure, and maintainable designs.
 Operational Excellence	Monitoring, alerting, automation, and incident handling.
 Security Mindset	RBAC, network policies, secrets management, least privilege.
 Communication	Structured explanation, assumptions, trade-offs, and alternatives.

★ TIP: Show how you think, not just what you know. Explain your thought process.

5. INTERVIEW TIPS

- Take a moment: Understand the question before answering.
- Think out loud: Share your thought process.
- Use real examples: Reference incidents or projects.
- Be structured: Use steps, diagrams, and architecture views.
- Clarify assumptions: Confirm when requirements are unclear.
- Stay practical: Focus on simplicity and reliability.
- Know your tools: kubectl, Helm, Prometheus, Argo CD, etc.
- Keep learning: Kubernetes evolves. Stay updated.

★ TIP: Confidence comes from preparation and experience. Keep practicing and ship systems.

6. SENIOR ENGINEER MINDSET

-  Own the reliability.
-  Think in systems.
-  Automate everything.
-  Observe and measure.
-  Collaborate and communicate.
-  Keep improving.

REMEMBER

Kubernetes is easy to install. Hard to run. Impossible to run well without experience. Be the engineer who makes it look easy.

★ TIP: Build, break, fix, and repeat. That is how senior engineers are made.



kubernetes

VERIQTA

PAGE 40

40. KUBERNETES PRODUCTION READINESS CHECKLIST



A production-ready cluster is secure, reliable, observable, and resilient.

Use this checklist before you go live.



AT A GLANCE

- ☒ Secure your cluster.
- ☒ Make it highly available.
- ☒ Observe everything.
- ☒ Optimize performance.
- ☒ Automate and test.
- ☒ Document and improve.

1. CLUSTER CHECKLIST

- ☐ Kubernetes version is supported and up to date.
- ☐ Control plane components are highly available.
- ☐ All nodes are healthy and monitored.
- ☐ etcd is backed up regularly and tested.
- ☐ RBAC is enabled and properly configured.
- ☐ Namespace isolation is enforced.
- ☐ Resource requests and limits are set.
- ☐ Admission controllers are enabled.
- ☐ Cluster autoscaler is configured.
- ☐ Backups and disaster recovery are tested.
- ☐ Cluster documentation is up to date.



2. SECURITY CHECKLIST

- ☐ RBAC is enforced with least privilege.
- ☐ Service Accounts are used correctly.
- ☐ Pod Security Standards / OPA is enabled.
- ☐ Network Policies are applied.
- ☐ Secrets are encrypted and externalized.
- ☐ Image scanning is enabled in CI/CD.
- ☐ Only signed and trusted images are allowed.
- ☐ Audit logging is enabled.
- ☐ etcd encryption is enabled.
- ☐ Control plane is not publicly accessible.
- ☐ Regular security scans are performed.



3. NETWORKING CHECKLIST

- ☐ CNI plugin is healthy and up to date.
- ☐ Pod-to-Pod communication is working.
- ☐ Network Policies are defined and enforced.
- ☐ Ingress Controller is configured.
- ☐ TLS is enabled for all external traffic.
- ☐ Load balancer is provisioned correctly.
- ☐ DNS is reliable and highly available.
- ☐ CoreDNS is monitored.
- ☐ No unnecessary ports are open.
- ☐ VPC / Subnets are properly designed.



4. STORAGE CHECKLIST

- ☐ Default StorageClass is defined.
- ☐ PersistentVolumes are monitored.
- ☐ PVCs have appropriate size and access modes.
- ☐ Backups of stateful data are automated.
- ☐ CSI drivers are installed and healthy.
- ☐ Storage performance is monitored.
- ☐ Snapshots are enabled for critical workloads.
- ☐ Encryption at rest is enabled.
- ☐ Data retention policies are defined.
- ☐ Restore procedures are tested.



5. MONITORING CHECKLIST

- ☐ Prometheus is installed and configured.
- ☐ All critical components are scraped.
- ☐ Grafana dashboards are in place.
- ☐ Alert rules are defined and tested.
- ☐ Logs are collected and centralized.
- ☐ Node metrics are monitored.
- ☐ Application metrics are exposed.
- ☐ SLOs / SLIs are defined.
- ☐ Alerts are routed to the right team.
- ☐ Monitoring stack is highly available.



6. HIGH AVAILABILITY CHECKLIST

- ☐ Control plane runs on 3 or more nodes.
- ☐ etcd runs on odd number of nodes (3/5/7).
- ☐ Worker nodes spread across AZs.
- ☐ Multi-AZ load balancer is configured.
- ☐ Cluster autoscaler is enabled.
- ☐ PodDisruptionBudgets (PDB) are configured.
- ☐ Readiness and liveness probes are set.
- ☐ Critical add-ons run with HA.
- ☐ Backups are stored in a different AZ/Region.
- ☐ Failure scenarios are tested regularly.



7. FINAL VERIQTA PRODUCTION PRINCIPLES



SECURE BY DEFAULT

Security is not optional.
Build it in from day one.



DESIGN FOR FAILURE

Assume failure.
Plan for it.
Test for it.



OBSERVE EVERYTHING

If you can't see it,
you can't fix it.
Monitor end to end.



AUTOMATE EVERYTHING

No manual steps.
Automate, test,
and repeat.



OPTIMIZE

Use resources wisely.
Optimize cost and performance.



DOCUMENT & IMPROVE

Document decisions.
Review regularly.
Always improve.